Минобрнауки России

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Волгоградский государственный технический университет»

Факультет электроники и вычислительной техники

Кафедра САПР и ПК

Согласовано
_____
(должность гл. специалиста предприятия)

_____ _____
(подпись) (инициалы, фамилия)

«_____» _____20___г

Утверждаю
Зав. кафедрой

_____ _____
(подпись) (инициалы, фамилия)

«_____» _____20___г

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к выпускной квалификационной работе бакалавра на тему
(наименование вида работы)
Разработка интеллектуального инструментария для автоматизированного создания динамических игровых миров

Автор _____ Коваленко Кирилл Дмитриевич
(подпись и дата подписания) (фамилия, имя, отчество)

Обозначение _____
(код документа)

Группа ИВТ-465
(шифр группы)

Направление 09.03.01 Информатика и вычислительная техника
(код и наименование направления, наименование программы (профиля))

Профиль «Разработка игровых и XR приложений»

Руководитель работы _____ _____
(подпись и дата подписания) (инициалы и фамилия)

Консультанты по разделам:

_____ _____ _____
(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

_____ _____ _____
(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

_____ _____ _____
(краткое наименование раздела) (подпись и дата подписания) (инициалы и фамилия)

Нормоконтролер _____ _____
(подпись, дата подписания) (инициалы и фамилия)

Волгоград 2027 г.

АННОТАЦИЯ

Объём пояснительной записки — 77 с., 16 рис., 13 табл., 38 источников, 4 приложения.

Выпускная квалификационная работа посвящена разработке подключаемого инструментария WorldWeaver для автоматизированного создания рельефа и интеллектуальных неигровых персонажей в Unity. Инструментарий объединяет интерпретацию естественного языка, процедурную генерацию Terrain, извлечение индивидуальных знаний персонажа и исполнение высокоуровневых действий после детерминированной проверки.

Практический результат представляет собой UPM-пакет с отдельными сборками Runtime и Editor, демонстрационную сцену, автоматические тесты и воспроизводимый набор вопросов к базе знаний. Локальные вычисления выполняются через Ollama. Для облачной демонстрации используется бесплатный Pollinations Classic API, поэтому для основного сценария не требуется оплата модельных запросов. Результаты эксперимента характеризуют конкретный корпус, версии моделей и условия запуска; они не переносятся автоматически на произвольные игровые проекты.

Ключевые слова: UNITY, ПРОЦЕДУРНАЯ ГЕНЕРАЦИЯ, NPC, БОЛЬШАЯ ЯЗЫКОВАЯ МОДЕЛЬ, RAG, EMBEDDINGS, ОРКЕСТРАЦИЯ, ВАЛИДАЦИЯ ДЕЙСТВИЙ.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ..... 5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ..... 8
1.1 Процедурное создание игровых миров..... 8
1.2 Разделение авторинга и поведения во время игры..... 9
1.3 Языковые модели и извлечение знаний..... 10
1.4 Интеллектуальные неигровые персонажи..... 11
1.5 Сопоставление существующих решений..... 12
1.6 Постановка задачи и критерии качества..... 14
1.7 Сопоставление с университетскими исследованиями..... 15
2 ПРОЕКТИРОВАНИЕ ИНСТРУМЕНТАРИЯ..... 17
2.1 Требования и границы системы..... 17
2.2 Структура Unity Package..... 18
2.3 Маршрутизация модельных задач..... 19
2.4 Абстракция провайдеров и сетевой контракт..... 20
2.5 Архитектура знаний и памяти..... 22
2.6 Спецификация и генерация Terrain..... 24
2.7 Модель решения и система действий..... 26
2.8 Диаграммы процессов и потоков данных..... 28
2.9 Выбор локальной среды и ограничения развёртывания..... 31
3 РЕАЛИЗАЦИЯ И ДЕМОНСТРАЦИЯ..... 34
3.1 Среда разработки и состав поставки..... 34
3.2 Окно настройки в Unity Editor..... 35
3.3 Процедурный генератор рельефа..... 37
3.4 Локальные и бесплатные облачные запросы..... 38
3.5 Реализация retrieval и индексации..... 40
3.6 Обработка диалога и структурированного ответа..... 42
3.7 Передача предмета и автономное движение..... 43
3.8 Диагностика и ограничения безопасности..... 45
3.9 Демонстрационный сценарий..... 46
4 ТЕСТИРОВАНИЕ И ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА..... 48
4.1 Цели и организация проверки..... 48
4.2 Модульные и сетевые испытания..... 49
4.3 Интеграция в Play Mode..... 51
4.4 Метрики качества ответов и retrieval..... 54
4.5 Локальное и облачное исполнение..... 57
4.6 Производительность и воспроизводимость Terrain..... 59
4.7 Формализация метрик и проверка воспроизводимости..... 60
4.8 Ограничения экспериментального вывода..... 62
ЗАКЛЮЧЕНИЕ..... 64
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ..... 66
ПРИЛОЖЕНИЕ А Техническое задание..... 71
ПРИЛОЖЕНИЕ Б Приемочные сценарии..... 72
ПРИЛОЖЕНИЕ В Корпус вопросов к индивидуальным знаниям..... 74
ПРИЛОЖЕНИЕ Г Порядок установки и повторения проверки..... 76

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ

Таблица 1 — ПЕРЕЧЕНЬ СОКРАЩЕНИЙ

Сокращение	Значение
API	Программный интерфейс взаимодействия компонентов
LLM	Большая языковая модель
NPC	Неигровой персонаж
RAG	Генерация ответа с извлечением внешних знаний
UPM	Unity Package Manager
JSON	Текстовый формат структурированных данных
fBm	Фрактальная сумма шумовых октав
NavMesh	Навигационная сетка для поиска пути
DTO	Объект передачи структурированных данных
UML	Унифицированный язык моделирования
DFD	Диаграмма потоков данных

ВВЕДЕНИЕ

Создание игрового мира включает несколько различных по природе задач. Рельеф и размещение объектов требуют управления геометрией и повторяемыми параметрами. Диалог персонажа требует интерпретации языка и согласования реплик с ролью. Игровые последствия разговора, например передача предмета или выбор цели перемещения, подчиняются строгим правилам состояния сцены. При раздельной реализации этих подсистем разработчик вынужден соединять несколько инструментов и самостоятельно поддерживать общий контекст.

Большие языковые модели позволяют формулировать намерения на естественном языке и получать содержательные ответы без предварительного описания всех диалоговых ветвей. Однако текстовый ответ сам по себе не обеспечивает корректного поведения игры. Модель может назвать отсутствующую цель, предложить передачу недоступного предмета либо использовать сведения, которые персонажу неизвестны. Поэтому практическая применимость LLM определяется не только качеством речи, но и устройством границы между вероятностным решением и детерминированным игровым кодом.

В этой работе рассматривается инструментальная задача: разработчик устанавливает пакет в существующий Unity-проект, формирует рельеф по описанию, добавляет компонент персонажа, подключает индивидуальную базу знаний и выбирает модель для диалога или планирования. Результатом является расширение среды разработки и набор runtime-компонентов. Демонстрационная игра служит средством проверки этого инструментария и не определяет архитектуру продукта целиком.

Актуальность задачи связана с распространением как облачных, так и локальных LLM, а также с потребностью контролировать стоимость, задержку и зависимость от поставщика. Анализ современных решений показывает, что разговорные персонажи, retrieval и действия уже представлены в готовых продуктах. Следовательно, обоснование разработки

не может строиться на утверждении об отсутствии таких инструментов. Исследовательский интерес представляет согласованное соединение этих возможностей с типизированной генерацией Terrain и проверяемым исполнением действий внутри переносимого пакета.

Цель работы — разработать и экспериментально оценить подключаемый инструментарий Unity, который преобразует естественно-языковые запросы в параметры игрового мира и контекстно-зависимые решения NPC, сохраняя детерминированный контроль над изменениями сцены.

Для достижения цели решаются следующие задачи: анализ возможностей современных средств Unity AI и интеллектуальных персонажей; формирование требований к переносимому пакету; проектирование маршрутизации моделей и границ доверия; разработка процедурного генератора рельефа; реализация индивидуального RAG и памяти NPC; разработка реестра действий и проверок предусловий; создание интерфейса Editor и демонстрационной сцены; проведение автоматических испытаний и сравнительного эксперимента.

Объект исследования — процессы автоматизации создания и изменения динамического игрового мира. Предмет исследования — архитектурные и алгоритмические способы объединения LLM, retrieval и процедурных компонентов Unity при ограниченном доступе модели к состоянию игры.

Исследование уточняется тремя вопросами. RQ1: как добавление индивидуального RAG изменяет совпадение известного факта в ответе одной и той же локальной модели? RQ2: какие полные и провайдерские задержки наблюдаются у выбранных локальных и бесплатной облачной конфигураций? RQ3: сохраняет ли детерминированный исполнительный слой проверяемые инварианты Unity при ошибочном или недопустимом решении модели? Первый вопрос проверяется сравнением QA без RAG и с

RAG, второй — измерением p50 и p95, третий — программными и сквозными испытаниями.

Методы работы включают анализ первичных технических источников, объектно-ориентированное проектирование, процедурную генерацию, векторное представление текста, поиск по косинусному сходству, контрактную валидацию и экспериментальное сравнение конфигураций. Программные свойства проверяются средствами Unity Test Framework, а влияние базы знаний оценивается на фиксированном авторском корпусе с известными ответами.

Практическая значимость состоит в возможности повторно использовать готовую архитектуру в учебных и экспериментальных Unity-проектах. Пакет предоставляет единый процесс настройки рельефа, моделей, знаний и действий; разработчик может заменить провайдера или добавить действие через интерфейс. При этом работа не претендует на замену профессиональных генераторов окружения или на доказательство художественной убедительности персонажей в масштабной игре.

Работа состоит из четырёх глав. В первой главе анализируются предметная область и аналоги. Во второй главе обосновывается архитектура. Третья глава описывает реализацию и демонстрационный сценарий. Четвёртая глава содержит методику испытаний, измерения и ограничения результатов. В приложениях приведены техническое задание, сценарии проверки и дополнительные диаграммы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Процедурное создание игровых миров

Процедурная генерация формирует игровое содержимое алгоритмически, используя параметры и начальные условия. Для рельефа таким содержимым выступает поле высот, для растительности — набор позиций и типов объектов, для правил игры — структура допустимых состояний. При одинаковых входных данных детерминированный алгоритм позволяет восстановить результат и исследовать влияние отдельного параметра [32].

Поле высот удобно для ландшафта, поверхность которого можно представить функцией h(x,z). Unity Terrain хранит такую поверхность в регулярной сетке и предоставляет операции обновления высот. Каждая ячейка содержит нормированное значение, а размер TerrainData определяет физический масштаб сцены [3]. У этого представления есть ограничения: нависающие скалы, пещеры и произвольная многослойная геометрия требуют дополнительных мешей. Поэтому генерация Terrain не равнозначна генерации любого трёхмерного мира.

Шумовые методы позволяют получать непрерывные неоднородные поверхности с небольшим числом параметров. Суммирование нескольких октав изменяет вклад крупных и мелких деталей. Амплитуда очередной октавы задаётся persistence, частота — lacunarity. Небольшое число октав удобно для интерактивного редактора, поскольку объём вычислений заранее ограничен разрешением heightmap и количеством проходов.

Одного шума недостаточно для выполнения содержательного описания, например требования о долине с озером. Необходимо ввести семантические преобразования поверхности: горный край, центральное понижение, направленный уклон. Эти преобразования связывают привычные пользователю слова с контролируемыми числовыми параметрами. В таком

устройстве генератор остаётся воспроизводимым, а естественный язык определяет спецификацию, а не каждую высоту.

Профессиональные средства, включая продукты Procedural Worlds, решают более широкий круг задач окружения: подготовку ландшафта, распределение объектов и художественную настройку [10]. Учебный инструментарий рассматривает ограниченный набор форм, достаточный для проверки архитектуры. Сравнение по количеству биомов или качеству готовой графики не является целью работы.

1.2 Разделение авторинга и поведения во время игры

В игровых инструментах необходимо различать помощь разработчику и интеллект персонажа в запущенной сцене. Помощник Editor может анализировать проект, создавать код или ассеты и управлять инструментами редактора. Runtime-персонаж должен реагировать на текущего игрока, учитывать игровые данные и выполнять действия в пределах возможностей, предоставленных разработчиком.

Это различие влияет на права доступа и требования к производительности. Во время разработки допустимы сравнительно продолжительные операции и ручная проверка изменений. Во время игры обработка запроса не должна блокировать кадр, а неправильный ответ должен приводить к контролируемому отказу. Исполнение произвольного сгенерированного кода не требуется ни для передачи предмета, ни для выбора конечной точки пути.

Современный Unity AI нельзя описывать как систему, поддерживающую только собственные модели Unity. Проверенная документация AI Gateway прямо предусматривает подключение сторонних агентов и выбор соответствующих моделей. Для Gateway и MCP описаны требования к подписке организации, назначению места и привязке проекта [5]. Эти сведения характеризуют интеграцию ассистента разработки; из них

нельзя автоматически вывести наличие или отсутствие готового runtime-контура индивидуальных знаний и действий NPC.

Средства MCP для Unity также относятся прежде всего к автоматизации разработчика. Например, Coplay MCP предоставляет операции над сценами, ассетами, скриптами и испытаниями [31]. Такой инструмент полезен при создании проекта, однако архитектурно отличается от поставляемого игрового компонента, который должен работать после завершения процесса разработки.

1.3 Языковые модели и извлечение знаний

Языковая модель формирует ответ на основе параметров и переданного контекста. Её обучение не содержит авторский лор нового игрового мира, если этот лор не был включён в запрос или специально использован при адаптации модели. Поэтому свободный диалог о конкретной деревне, её жителях и предметах требует внешнего источника знаний.

Подход RAG соединяет генеративную модель с извлекаемой внешней памятью. В исходной работе Lewis и соавторов рассматривается сочетание параметрического генератора и непараметрического компонента извлечения для задач, требующих знаний [14]. В игровом инструментарии этот принцип реализуется как поиск нескольких релевантных фрагментов авторского корпуса и добавление их в контекст NPC.

Типовой pipeline включает загрузку документа, разбиение текста, получение embedding-вектора, сохранение фрагмента вместе с метаданными, представление запроса тем же embedding-пространством и поиск ближайших векторов. После retrieval генератор получает исходную реплику, сведения о персонаже и найденные фрагменты. Наличие RAG не отменяет возможности ошибки: неверно найденный фрагмент или неправильная интерпретация могут привести к недостоверному ответу.

Важен выбор границы фрагмента. Слишком короткий chunk теряет контекст факта; слишком длинный увеличивает объём нерелевантного

текста. Перекрытие соседних окон снижает риск разрыва факта на границе, но создаёт частично дублирующиеся результаты. Поэтому метрики на уровне фрагментов и документов должны различаться: присутствие правильного документа в top k не означает, что все k фрагментов полезны.

Индивидуальность персонажа требует ограничения корпуса. Если торговец не знает пароль стражи, один только высокий cosine score не должен открывать ему служебный документ. Фильтрация по Scope должна предшествовать ранжированию. Передача всех найденных документов модели с инструкцией «не использовать чужие знания» оставляет ограничение на вероятностном уровне и не обеспечивает изоляцию данных.

Историю разговора также нецелесообразно полностью добавлять в каждый запрос. Её размер растёт с продолжительностью игры, а большая часть старых реплик может быть не связана с текущим вопросом. Извлечение релевантной памяти позволяет ограничивать контекст, но требует решения о сроке хранения и достоверности события. Запись «модель предложила передачу» отличается от записи «передача исполнена». В этой работе эпизодическое событие создаётся после результата валидатора.

1.4 Интеллектуальные неигровые персонажи

Для традиционного NPC поведение обычно задаётся конечным автоматом, деревом поведения или другими заранее определёнными правилами. Такие методы хорошо контролируются и подходят для реакций с жёсткими временными ограничениями. Однако описание всех возможных формулировок свободного разговора становится трудоёмким, особенно при индивидуальных характерах и знаниях.

Генеративный агент расширяет эту модель за счёт языковой интерпретации, памяти и планирования. Park и соавторы показывают архитектуру, в которой наблюдения, извлечение памяти и планирование участвуют в формировании поведения [15]. Для рассматриваемого проекта полезен сам принцип разделения памяти и решения, но результаты

социальной симуляции из этой работы не считаются доказательством качества разработанного NPC.

Работа ReAct связывает рассуждение языковой модели с действиями во внешней среде [16]. Применительно к Unity это означает выбор дискретного высокоуровневого действия, например MoveTo или GiveItem. Низкоуровневое движение остаётся задачей NavMeshAgent, а состояние предметов — задачей Inventory. Модель не должна определять координату Transform каждый кадр: это увеличивает задержку, стоимость и нестабильность управления.

Личность персонажа влияет на выбор среди допустимых решений. Торговец может отказать, попросить оплату или согласиться помочь. Но ограничения игры нельзя выражать только характеристикой «осторожный». Если NPC не должен отдавать больше одного предмета, это правило задаётся числовым лимитом и проверяется кодом. Текстовые установки личности остаются мягкими предпочтениями, а не доказанной политикой доступа.

Существенный риск связан с расхождением речи и действия. Реплика «я отдал кристалл» не означает, что предмет действительно переместился. Поэтому ответ модели должен содержать отдельное поле речи и массив структурированных действий. Пользовательский интерфейс отображает подтверждённое состояние инвентаря, а ошибочное действие фиксируется с причиной отказа.

1.5 Сопоставление существующих решений

Сравнение выполнено по официальной документации и репозиториям на дату исследования. Оно характеризует опубликованные возможности, но не заменяет установку и нагрузочное испытание всех коммерческих SDK. Для неподтверждённой функции используется формулировка «не установлено», поскольку отсутствие сведений в одном разделе не доказывает отсутствие возможности во всём продукте.

Таблица 2 — Сопоставление существующих решений

Решение	**Основная подтверждённая возможность**	**Существенное отличие от задачи работы**
Unity AI Gateway	Сторонние агенты и помощь в Editor	Рассматривается отдельно от runtime-контура NPC
Convai Unity SDK	Персонажи, знания, действия и цели	Готовая специализированная экосистема персонажей
Inworld Runtime	LLM, речь, embeddings и knowledge	Набор runtime-модулей с собственной архитектурой
NVIDIA ACE	Agent, Chat, RAG и on-device inference	C/C++ и собственные требования inference-стека
Promethean AI	Поиск и компоновка 3D-ассетов	Работа с окружением, не только с heightmap
Gaia	Процедурное Unity-окружение	Более широкие художественные средства Terrain
LLMUnity	Локальные модели, диалог и RAG	Уже решает существенную часть локального NPC
WorldWeaver	TerrainSpec, scoped RAG и проверенные действия	Ограниченный учебный набор с явной маршрутизацией

Convai предоставляет средства конфигурирования доступных персонажу действий, объектов и целей, а также локального исполнения [6]. Следовательно, исполнение действий NPC не является самостоятельной новизной проекта. Официальное сообщение Inworld указывает на вывод прежнего Character Studio из эксплуатации и перенос соответствующих возможностей в Agent Runtime SDK и Unity-шаблоны с LLM, речевыми компонентами, knowledge и memory [7]. Поэтому legacy Studio не рассматривается как действующий продукт. Эти продукты необходимо сравнивать по составу и интеграции, а не представлять как обычные чат-боты.

NVIDIA ACE Game Agent SDK описывает Agent, Chat и RAG API, а также отдельные средства локального inference [8]. На используемом Apple M4 не выполнялась проверка NVIDIA inference-стека, поэтому работа не делает утверждения о его производительности на данной машине. Promethean

AI ориентирован на использование и организацию ассетов; его сайт называет Unreal основной интеграцией и описывает плагины других редакторов [9].

LLMUnity особенно важен для сопоставления, поскольку предоставляет локальную LLM-интеграцию и RAG непосредственно в Unity [11]. Разработка собственного компактного backend в настоящей работе мотивируется исследованием контролируемого pipeline и переносимости HTTP-провайдеров. Это инженерный выбор, а не вывод о непригодности готовой библиотеки.

Общие RAG-frameworks LangChain и LlamaIndex дают развитые абстракции загрузчиков, индексов и vector store [12, 13]. Их подключение через отдельный сервис расширило бы возможности системы, но добавило бы процесс и язык исполнения. Для малого демонстрационного корпуса выбран C#-индекс с интерфейсом замены backend. При увеличении объёма данных этот выбор должен быть пересмотрен.

1.6 Постановка задачи и критерии качества

По результатам анализа задача сформулирована как создание переносимого инструментария с ограниченной и измеряемой областью ответственности. Требуются единое окно Editor, генерация Terrain по типизированной спецификации, настройка NPC, изоляция знаний, несколько модельных маршрутов и проверка действий. Качество определяется воспроизводимой установкой и корректностью сценария, а не количеством перечисленных технологий.

Функциональная корректность включает воспроизведение Terrain при одном seed, отсутствие изменений инвентаря при неверной команде, получение ответа через реальную модель, корректную изоляцию знаний и исполнение движения через NavMesh. Надёжность включает отмену запроса, timeout, ограниченный retry и продолжение работы редактора после контролируемого отказа.

Исследовательские вопросы RQ1–RQ3 связывают индивидуальные знания с совпадением факта, модельную конфигурацию с наблюдаемой задержкой и исполнительный слой с сохранением инвариантов. Для каждого вопроса требуется собственный источник доказательств: QA-корпус, временной протокол или тест состояния. Эксперимент не доказывает общую интеллектуальность NPC и не измеряет удовольствие игрока от диалога.

1.7 Сопоставление с университетскими исследованиями

Опубликованные выпускные исследования полезны для выбора метода оценки, но не задают единый шаблон оформления и не являются источником готового авторского результата. В бакалаврской работе O. Kyzr о процедурной генерации внешних сцен отдельно рассмотрены реализация, производительность и качественное воспроизведение сцен [35]. Для настоящей работы существенно такое разграничение: измерение функции heightmap не заменяет оценки времени подготовки полного окружения.

Магистерская работа F. Marcher посвящена инструменту генерации Terrain в Unity и включает отдельное исследование использования интерфейса участниками [36]. Из этого следует методическое требование различать функциональную проверку Editor и оценку удобства. В WorldWeaver проведён проход основных операций разработчиком и проверена чистая установка. Эти сведения подтверждают исполнимость сценария; на их основании не выводятся удобство для новичков и процент сокращения труда.

Бакалаврская работа O. R. J. Wakeford исследует генерацию рельефа с фиксированными точками [37]. Она показывает необходимость явно описывать геометрические ограничения и разные виды оценки. Для WorldWeaver аналогичное требование состоит в точном определении семейства TerrainSpec: радиальный край и долина не означают поддержку любого описанного мира. Численные показатели разных работ не

сравниваются напрямую, поскольку различаются алгоритмы, устройства и измерительные границы.

Исследование P. Aslan о LLM-персонажах отдельно рассматривает несколько сторон RAG, включая работу при отсутствии знаний и противоречивой информации [38]. Наш набор из 30 вопросов с известными ответами покрывает только более узкую задачу. Он не содержит полноценного отрицательного корпуса для оценки отказа и контрфактической устойчивости. Эти направления остаются за пределами подтверждённых метрик.

Сопоставление показывает, что полноту ВКР определяет связь постановки задачи, реализации и проверяемого результата. Вклад WorldWeaver является интеграционным: типизированные спецификации, scope знаний, маршрутизация по задачам, реестр действий и единый процесс работы через UPM. RAG, fBm и NavMesh используются как известные методы, а не объявляются изобретёнными в настоящей работе.

Вывод первой главы состоит в необходимости разделить генерацию намерения и исполнение последствий. Современные аналоги подтверждают практическую применимость LLM и RAG в играх, но также задают требования к честному сопоставлению. Для дальнейшей разработки выбрана архитектура, в которой модель заменяема, знания имеют область видимости, а состояние Unity меняется только после проверки контрактов.

2 ПРОЕКТИРОВАНИЕ ИНСТРУМЕНТАРИЯ

2.1 Требования и границы системы

Исходное техническое задание определяет инструментарий как подключаемый продукт для Unity 6+, а не отдельную игру. Для реализации выбран UPM-пакет WorldWeaver с демонстрационным проектом. Пакет должен предоставлять авторинг в Editor и компоненты, которые продолжают выполнять свои функции в Play Mode.

Разработчик задаёт описание рельефа, параметры модели, личность NPC, документы знаний и список разрешённых действий. Игрок вводит текстовую реплику и наблюдает речь персонажа и последствия его решения. Внешние системы предоставляют только inference: текстовый completion или embedding-вектор. Они не получают программного интерфейса прямого изменения объектов Unity.

Обязательные функции ограничены текстовым взаимодействием, Terrain, retrieval, действиями и диагностикой. Голосовые интерфейсы и извлечение текста из PDF не включены в проверенный сценарий. Это ограничение не скрывается под общим утверждением о поддержке любых источников. Импорт знаний принимает TXT, Markdown и JSON с установленным форматом.

Таблица 3 — Требования и границы системы

Группа требований	Проектное решение	Способ проверки
Переносимость	Отдельный UPM-пакет, сборки Editor и Runtime	Установка и компиляция в тестовом проекте
Управляемость Terrain	TerrainSpec и детерминированный C#	Повторная генерация по seed
Индивидуальные знания	Scope перед поиском	Тест исключения документа другого NPC
Управляемые последствия	NpcDecision и ActionDispatcher	Неверные amount, target и permissions
Несколько моделей	ModelTask и OrchestratorProfile	Проверка разных назначений маршрутов

Группа требований	Проектное решение	Способ проверки
Отказоустойчивость	Timeout, cancellation и bounded retry	Локальный HTTP-сервер ошибок
Диагностика	PipelineTrace и вкладка Debug	Сопоставление JSON, проверки и инвентаря

Нефункциональные требования связаны с сохранностью сцены и контролем ресурсов. Создание Terrain формирует собственный объект и не заменяет существующие ландшафты. Повторная генерация допускается только для объекта с маркером владения GeneratedTerrain. Локальные модели хранятся вне репозитория, а количество октав, размер ответа и число фрагментов имеют явные пределы.

2.2 Структура Unity Package

Пакет размещён в каталоге Packages/com.worldweaver.toolkit. Файл package.json определяет имя, версию и зависимость от официального пакета Newtonsoft JSON. Сборка WorldWeaver.Runtime содержит модели данных, providers, RAG, действия, компоненты NPC и Terrain. Сборка WorldWeaver.Editor содержит окно настройки, создание сцены и инструменты исследования.

Разделение сборок не позволяет runtime-коду зависеть от UnityEditor. Это важно для дальнейшего использования игровых компонентов за пределами редактора. Средства измерений, создание ассетов и графический авторинг остаются в Editor. Тесты вынесены в отдельные сборки EditMode и PlayMode и подключаются через testables демонстрационного проекта.

Unity Package Manager поддерживает Git-зависимости и выбор подкаталога пакета через параметр path [2]. Такая структура позволяет держать программный продукт, демонстрацию и документы в одном репозитории, сохраняя независимую точку установки. Для локальной разработки используется относительная file-зависимость; она не должна ошибочно публиковаться как универсальный сетевой адрес.

Взаимодействие основных компонентов показано на рисунке. Окно Editor обращается к авторингу рельефа и профилям; во время игры IntelligentNPC передаёт запрос в NpcPipeline. Pipeline использует retrieval, router и dispatcher, но не зависит от конкретного поставщика inference.

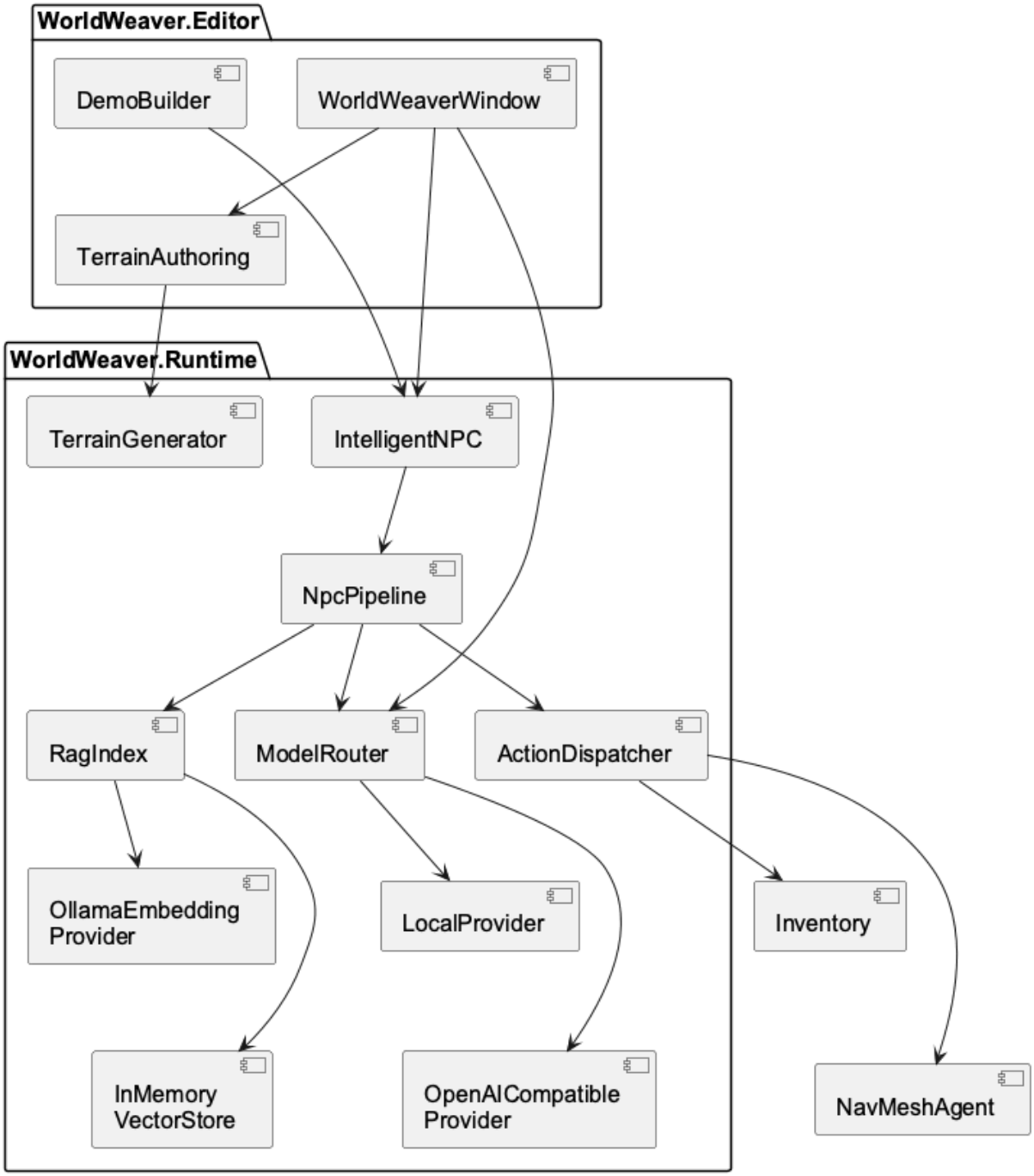


Рисунок 1 — Компоненты пакета WorldWeaver

2.3 Маршрутизация модельных задач

В системе выделены три текстовые задачи: Terrain, Dialogue и Planning. Для каждой OrchestratorProfile содержит отдельную ссылку на ModelProfile.

Метод Resolve проверяет наличие маршрута и возвращает явно назначенный профиль. Такая организация допускает разные модели и endpoints без изменения компонента NPC.

Terrain требует короткой интерпретации параметров. Dialogue формирует русскоязычную речь с учётом знаний и личности. Planning выбирает высокоуровневое действие на основе компактного состояния мира. Embeddings выделены в отдельный IEmbeddingProvider, поскольку их результат имеет другой тип и не должен обрабатываться как текстовый completion.

В смешанной демонстрационной конфигурации диалог выполняется бесплатной облачной моделью, а Terrain и Planning — локально. Причина связана с ограничением частоты анонимного API: отправлять в облако все вспомогательные задачи означало бы занимать редкие доступные интервалы запросов. Детерминированная проверка остаётся обычным C#-кодом и не требует ещё одной модели.

Такой router не является системой совместного обсуждения нескольких агентов. Он не осуществляет голосование или автоматическую оценку «умности» моделей. Его назначение — разделить задачи и сделать выбранный маршрут наблюдаемым. В текущей версии отказ провайдера не вызывает скрытого перехода на fixture или mock, поскольку это исказило бы демонстрацию и измерения.

Кэш экземпляров провайдеров внутри ModelRouter уменьшает число создаваемых HTTP-клиентов. Освобождение pipeline закрывает принадлежащие ему сетевые ресурсы. Перед сменой профиля демонстрационный интерфейс освобождает прежние services; смена модели становится явным действием разработчика или пользователя demo.

2.4 Абстракция провайдеров и сетевой контракт

Интерфейс ILLMProvider содержит CompleteAsync, HealthAsync и Dispose. CompletionRequest включает сообщения и JSON Schema ожидаемого

объекта. CompletionResult содержит текст, фактический provider/model, задержку и доступные счётчики токенов. При отсутствии метрики она не восстанавливается по предположению.

LocalProvider использует native API Ollama. Схема передаётся в format, streaming выключен, а для выбранной локальной модели отключается дополнительный thinking-вывод. Native API позволяет получить prompt_eval_count, eval_count и eval_duration. Эти данные нужны для измерения локальной скорости вывода и не подменяются скоростью сети [17].

OpenAICompatibleProvider реализует общий формат chat completions с отдельными особенностями Groq и Pollinations. Совместимость означает поддержку проверенного подмножества полей. Она не гарантирует идентичность всех API: путь получения моделей, имя параметра ограничения токенов и поддержка строгой схемы могут отличаться [21, 22].

ModelProfile хранит endpoint, model ID, timeout, число повторов, temperature и лимит токенов. Если нужен ключ, сериализуется только имя переменной окружения. Конструктор проверяет, что адрес использует HTTPS или локальный loopback HTTP, не содержит credentials, query и fragment. Автоматические redirects отключены, чтобы заголовок Authorization не передавался неожиданному хосту.

JsonTransport ограничивает объём ответа двумя мебибайтами. Чтение тела остаётся под действием timeout и cancellation token; иначе timeout только на получение заголовков не защитил бы от зависшего потока. Ошибки 400 и недоступная модель не должны повторяться как временная проблема. Для 429, 408 и серверных ошибок предусмотрены ограниченные повторы. Ответ 402 бесплатного API обрабатывается как временный предел только при наличии Retry-After; никакой платёжный механизм не вызывается.

Важна разница между транспортным успехом и прикладным. HTTP 200 подтверждает получение тела, но не валидность NpcDecision. Ответ проверяется ещё раз после десериализации. При JSON Object Mode схема

дополнительно описывается в system-сообщении, поскольку сам этот режим гарантирует только синтаксическую форму JSON, а не точные поля.

2.5 Архитектура знаний и памяти

KnowledgeBase представляет редактируемый ассет со списком KnowledgeDocument. Документ имеет идентификатор, Scope, заголовок, текст и MemoryKind. Chunk сохраняет идентификатор исходного документа, смещение, область видимости, текст и вектор. Такая структура обеспечивает связь найденного результата с авторским источником.

TextChunker использует окна по 500 символов с перекрытием 80 символов. Размер задан в символах, а не в токенах; это осознанное упрощение для короткого русского лора. Идентификатор chunk вычисляется через SHA-256 от scope, документа, смещения и содержимого. Повторная индексация одинакового текста даёт тот же идентификатор, а перенос текста другому персонажу меняет его.

Для получения векторов используется EmbeddingGemma. Модель предназначена для многоязычных текстовых представлений; в проверенном локальном запуске получены векторы размерности 768 [19]. Запрос и документ имеют разные префиксы, что соответствует поисковому назначению embedding-модели [20]. Документы не интерпретируются как инструкции разработчика.

Для векторов a и b косинусное сходство определяется как отношение скалярного произведения к произведению норм. Значение зависит от направления векторов, а не от их абсолютной длины. Нулевые векторы обрабатываются отдельно, а несовместимая размерность и нечисловые значения отклоняются. Изменение embedding-модели требует перестроения индекса, поскольку сравнение в разных пространствах не имеет корректной интерпретации.

InMemoryVectorStore сначала ограничивает множество документов по Scope и MemoryKind, затем вычисляет сходство и выбирает top k. При

равных score порядок стабилизируется идентификатором, чтобы исключить случайную перестановку результатов. Для N фрагментов размерности d стоимость одного полного поиска составляет O(Nd). Для демонстрационного корпуса эта сложность приемлема и проще для проверки, чем приближённый индекс.

RagIndex различает статические знания, эпизодические события и разговорную память. Для основной базы применяется настраиваемый top k, дополнительно извлекаются до двух эпизодических и двух разговорных фрагментов. При сборке контекста общий текст retrieved ограничивается 4000 символами. Это ограничение не является точным токеновым budget, поэтому максимальный размер всей prompt также зависит от описания личности и состояния мира.

Память ограничена 32 последними документами текущего сеанса. При превышении предела старый документ и соответствующие ему chunks удаляются. Индекс не является системой постоянного сохранения игры: завершение процесса уничтожает его содержимое. Статическая KnowledgeBase остаётся ассетом и может быть проиндексирована повторно.

Перестроение статического индекса выполняется через временное хранилище. Новый backend становится активным после успешной обработки всех документов и проверки cancellation token. Это защищает предыдущий индекс от частичной замены при отмене. Ошибка индексации памяти после уже исполненного действия фиксируется отдельно и не должна превращать произошедшую передачу предмета в фиктивный откат.

Для поиска используется косинусная мера

$$c(q,d) = \frac{\sum_{i=1}^{D} q_i d_i}{\sqrt{\sum_{i=1}^{D} q_i^2} \cdot \sqrt{\sum_{i=1}^{D} d_i^2}}$$

Здесь q — вектор запроса, d — вектор фрагмента, D = 768 — размерность представления проверенной модели. После фильтрации

результат представляет упорядоченный список пар «фрагмент, score». Значение score не является вероятностью истинности текста. Даже высокий показатель означает близость представлений, поэтому предметная достоверность должна оцениваться отдельно по ответу и источнику.

При N фрагментах и d координатах память самих векторов имеет порядок 4Nd байт для float32, без учёта строк и объектов среды выполнения. Например, для 1000 фрагментов размерности 768 числовая часть занимает примерно 3,07 МБ. Это расчёт объёма представления, а не измерение полного потребления процесса. Такая оценка объясняет выбор простого точного поиска для небольшого корпуса и необходимость другого backend при существенно большем N.

Граница Scope относится к доступу к данным, а не к качеству ранжирования. Если запрос совпадает со служебным паролем другого персонажа, этот документ всё равно исключается до вычисления сходства. Проверка scope входит как в формирование индекса NPC, так и в контракт поиска. Двойное ограничение защищает от случайного смешивания документов при дальнейшем расширении хранилища.

2.6 Спецификация и генерация Terrain

TerrainSpec содержит seed, физические размеры, разрешение, параметры fBm, силу горного края, глубину долины, уровень воды, южное понижение и число проходов сглаживания. Текущая версия использует один тип шума и не предоставляет произвольные карты биомов или гидравлическую эрозию. Выбор ограниченного набора параметров уменьшает неоднозначность JSON-контракта.

Таблица 4 — Спецификация и генерация Terrain

Поле	Смысл	Допустимая область
seed	Начальное условие шума	Целое число
width и length	Размер поверхности	От 32 до 2048 единиц Unity
height	Масштаб высоты	От 4 до 512

Поле	Смысл	Допустимая область
resolution	Размер регулярной сетки	33, 65, 129, 257 или 513
octaves	Число шумовых октав	От 1 до 8
mountainStrength	Выраженность горного края	От 0 до 1
valleyDepth	Центральное понижение	От 0 до 0,8
waterLevel	Нормированная высота воды	От 0 до 0,8
smoothingPasses	Проходы сглаживания	От 0 до 8

Валидация проверяет числовые типы и диапазоны до выделения карты. Например, дробное resolution не округляется в допустимое целое, а считается нарушением контракта. Это важно, поскольку автоматическое преобразование типов могло бы скрыть ошибку модели и затруднить сопоставление запроса с фактическим результатом.

Шумовая функция использует целочисленный hash координат и seed, сглаженную интерполяцию соседних значений и ограниченное число октав. После базового fBm к поверхности добавляется радиальный горный край и вычитается центральная долина. Множитель southSlope понижает южную часть края. Результат нормируется в диапазон, допустимый для TerrainData.

Сглаживание выполняется синхронно: каждый проход читает предыдущую карту и пишет новую. Это исключает зависимость результата от порядка обхода обновляемых ячеек. Используемый фильтр не моделирует перенос воды или осадка, поэтому в документации он не называется физической эрозией.

Существующий Terrain меняется только через TerrainAuthoring и маркер GeneratedTerrain. При Regenerate регистрируется Undo полного TerrainData, после чего обновляются высоты, размеры и связанная водная плоскость [25]. Создание нового объекта не затрагивает чужие TerrainData. Сохранённые ассеты генерации могут оставаться в Project после Undo создания объекта; их удаление не является частью отмены сцены.

Расчёт нормированного шума при O октавах имеет вид

$$F(u,v) = \frac{\sum_{o=0}^{O-1} a_o N(f_o u, f_o v, s_o)}{\sum_{o=0}^{O-1} a_o}$$

Амплитуда начинается с единицы и умножается на persistence после каждой октавы, частота начинается со scale и умножается на lacunarity. Seed октавы равен seed + 1013o с переполнением в установленной целочисленной арифметике. Такая конкретизация нужна для воспроизведения алгоритма, поскольку одно название fBm не определяет реализацию шума.

Финальная высота до ограничения диапазона получается суммой базового уровня 0,20, шумового вклада 0,24F, горного края с южным коэффициентом и отрицательного вклада долины. После этого применяется Clamp01. WaterLevel не изменяет поле высот: он определяет высоту дополнительной визуальной плоскости. Поэтому изменение уровня воды и изменение формы дна являются разными операциями.

Для внутренних ячеек проход сглаживания p выражается формулой

$$h^{p+1} = 0{,}65\, h^{p} + 0{,}35\, avg_4(h^{p})$$

Здесь avg_4 обозначает среднее четырёх соседей. Край массива в проходе сохраняется. Алгоритм является локальным усреднением и не моделирует перенос массы или потоки воды. При описании результатов термин «эрозия» для этой операции не используется.

2.7 Модель решения и система действий

Ответ NPC имеет два обязательных поля: speech и actions. Каждая команда содержит type, target, item и amount. Даже для действия без предмета поля сохраняются: item и target при необходимости равны пустой строке, amount равен нулю. Единый DTO упрощает совместимость разных JSON-режимов.

NpcDecision.Parse отклоняет неизвестные и дублирующиеся поля, неправильные типы, чрезмерно длинные строки и массивы более четырёх действий. Структурная схема полезна для генерации, но не заменяет

прикладные проверки. JSON Schema задаёт форму объекта, а право на изменение инвентаря определяется текущим миром [26].

ActionDispatcher хранит реестр IGameAction. Для встроенных операций используются TransferAction и WorldAction. Добавление нового обработчика через Register не требует изменения router. Пользовательский обработчик обязан выполнять Validate без изменения сцены; нарушение этого контракта делает общую предварительную проверку бессмысленной.

Перед исполнением решения dispatcher формирует словарь явно известных NPC сущностей. Неизвестный target и повторяющийся entity ID приводят к отказу. Для передачи предметов создаётся проекция каждого затронутого инвентаря. Она резервирует количество и вместимость до реального изменения объектов. Так два действия по шесть кристаллов не могут оба пройти при исходном остатке десять.

Количество передаваемых предметов ограничивается не только остатком, но и общим MaxTransferPerDecision. Проверяются дальность взаимодействия и разрешение TakeItem. Текстовое ограничение «не отдавай много» не считается заменой этих полей. Для перемещения проверяется наличие агента на NavMesh и полный путь к достижимой точке.

После успешного предконтроля действия выполняются последовательно без await в главном потоке. Это предотвращает изменение состояния между проверкой и записью из-за другого продолжения async-метода. Для встроенных инвентарных операций такой порядок обеспечивает согласованную передачу. Полный rollback произвольных пользовательских действий во внешних системах текущий интерфейс не гарантирует.

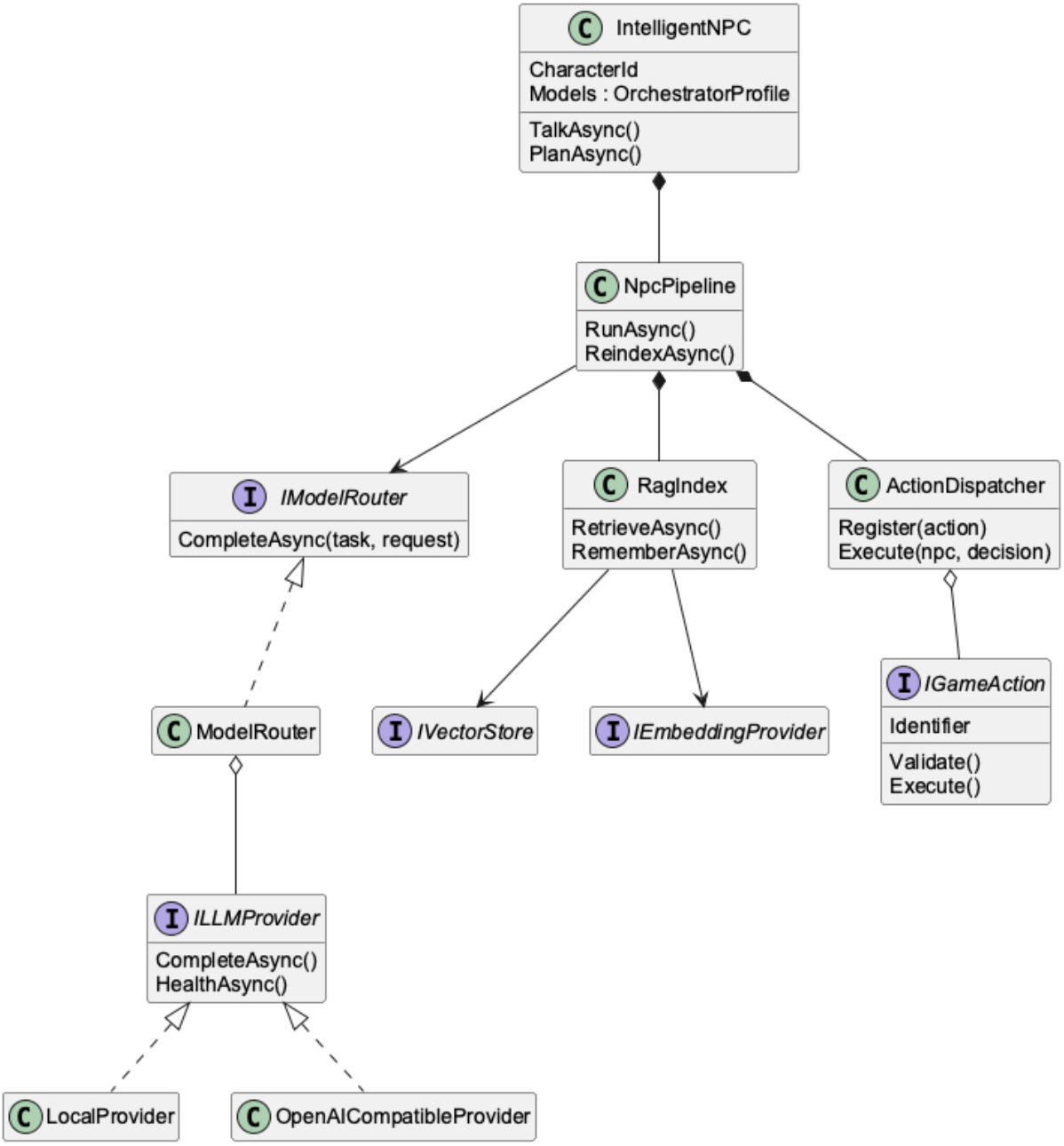


Рисунок 2 — Классы runtime и точки расширения

2.8 Диаграммы процессов и потоков данных

Диаграмма прецедентов разделяет действия разработчика и игрока. Разработчик настраивает пакет и исследует traces; игрок участвует в диалоге. Автономное планирование использует тот же контур проверки действий, поэтому отдельный неконтролируемый путь изменения мира не создаётся.

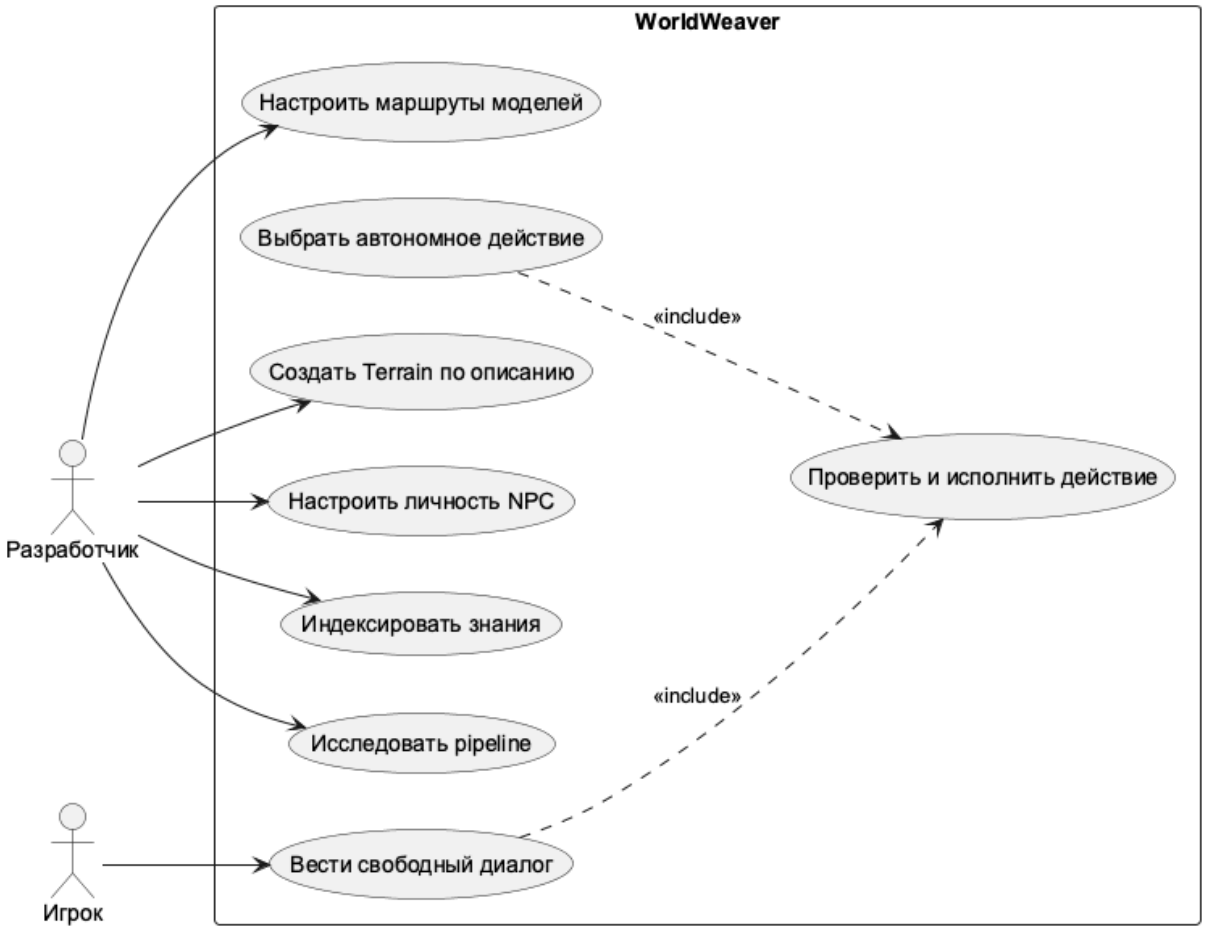


Рисунок 3 — Прецеденты использования инструментария

На диаграмме последовательности показан разговор с передачей предмета. Retrieval предшествует запросу модели, а инвентарь изменяется после разбора и валидации. Запись памяти содержит результат исполнения, а не одно лишь намерение модели.

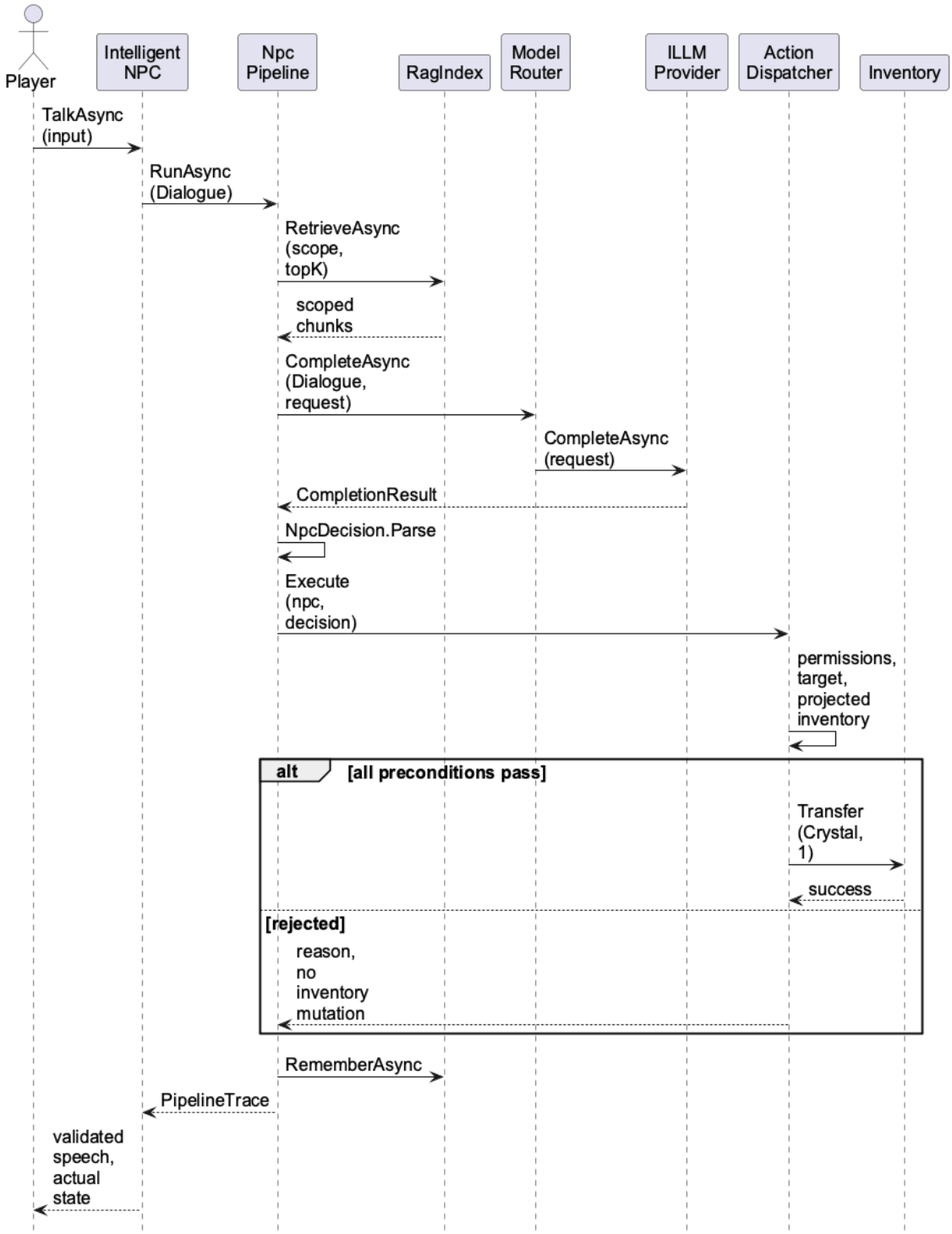


Рисунок 4 — Последовательность обработки диалога

Диаграмма потоков данных выделяет внешние inference-сервисы, хранилище знаний, состояние сцены и журнал. Через сетевую границу проходят текстовые данные и структурированные результаты. Внешний сервис не записывает Inventory или TerrainData непосредственно.

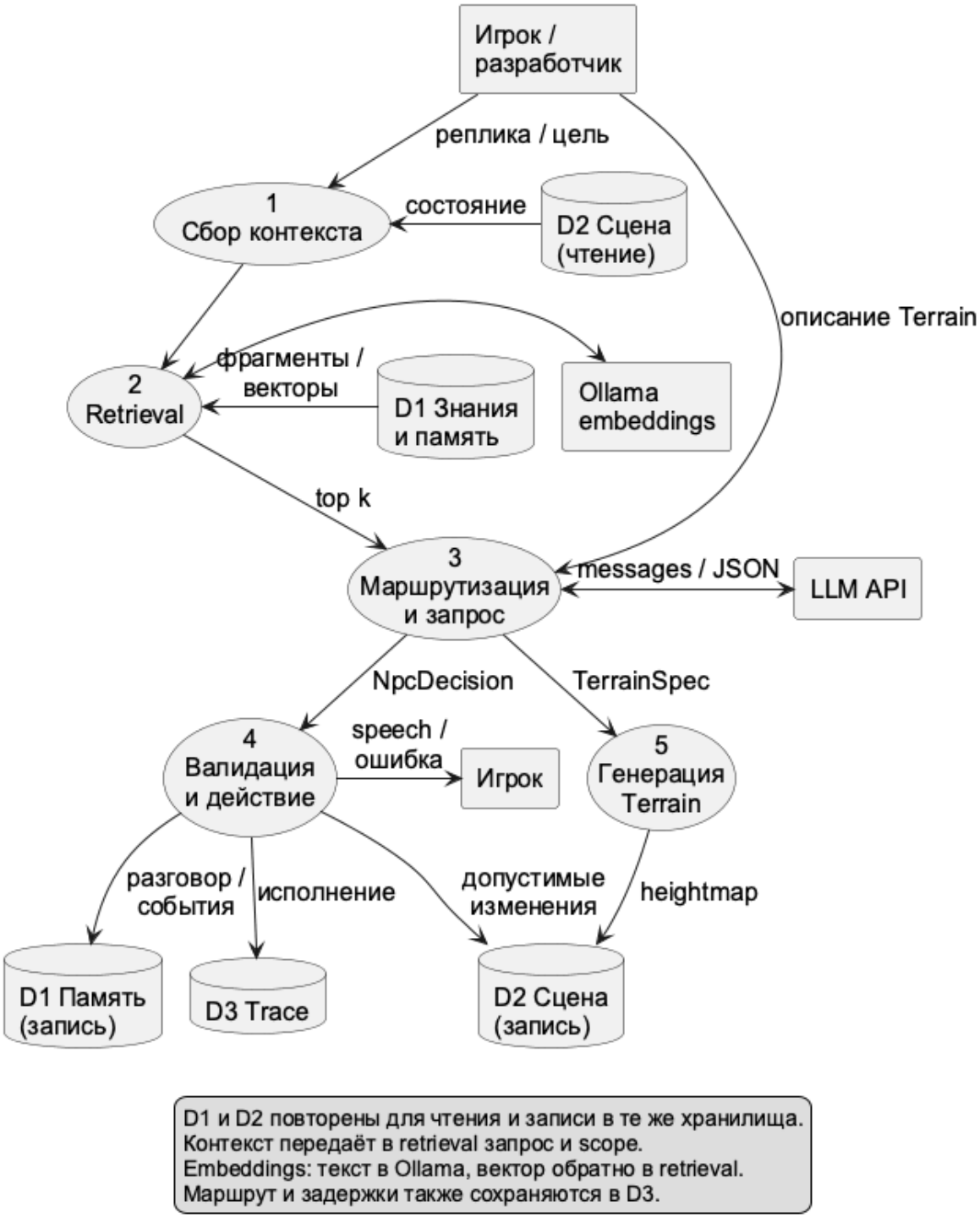


Рисунок 5 — Потоки данных WorldWeaver

2.9 Выбор локальной среды и ограничения развёртывания

Для локального inference рассмотрены Ollama, LM Studio и llama.cpp. Все три подхода позволяют организовать локальное исполнение, однако отличаются управлением моделями и API [17, 33, 34]. На машине уже был установлен Ollama, поэтому его использование уменьшило объём изменений

среды и позволило проверить реальные запросы через один локальный daemon.

Apple M4 имеет объединённую память, которую одновременно используют Unity, операционная система и inference. Установка модели на 120 миллиардов параметров не соответствует доступным 16 ГБ. Поэтому для коротких запросов Terrain и QA выбрана небольшая Qwen3.5 2B и отдельная embedding-модель; для итогового Dialogue и Planning дополнительно используется квантованная qwen2.5-coder 7,6B. Выбор основан на возможности совместного запуска, а не на попытке воспроизвести cloud-модель на любом оборудовании [18].

Бесплатный Pollinations API не требует ключа, но ограничивает частоту и не гарантирует доступность [23]. Это делает его пригодным для демонстрации и сравнительного опыта при соблюдении интервалов, но не позволяет обещать фиксированную задержку в коммерческой игре. Локальная конфигурация остаётся обязательным воспроизводимым вариантом.

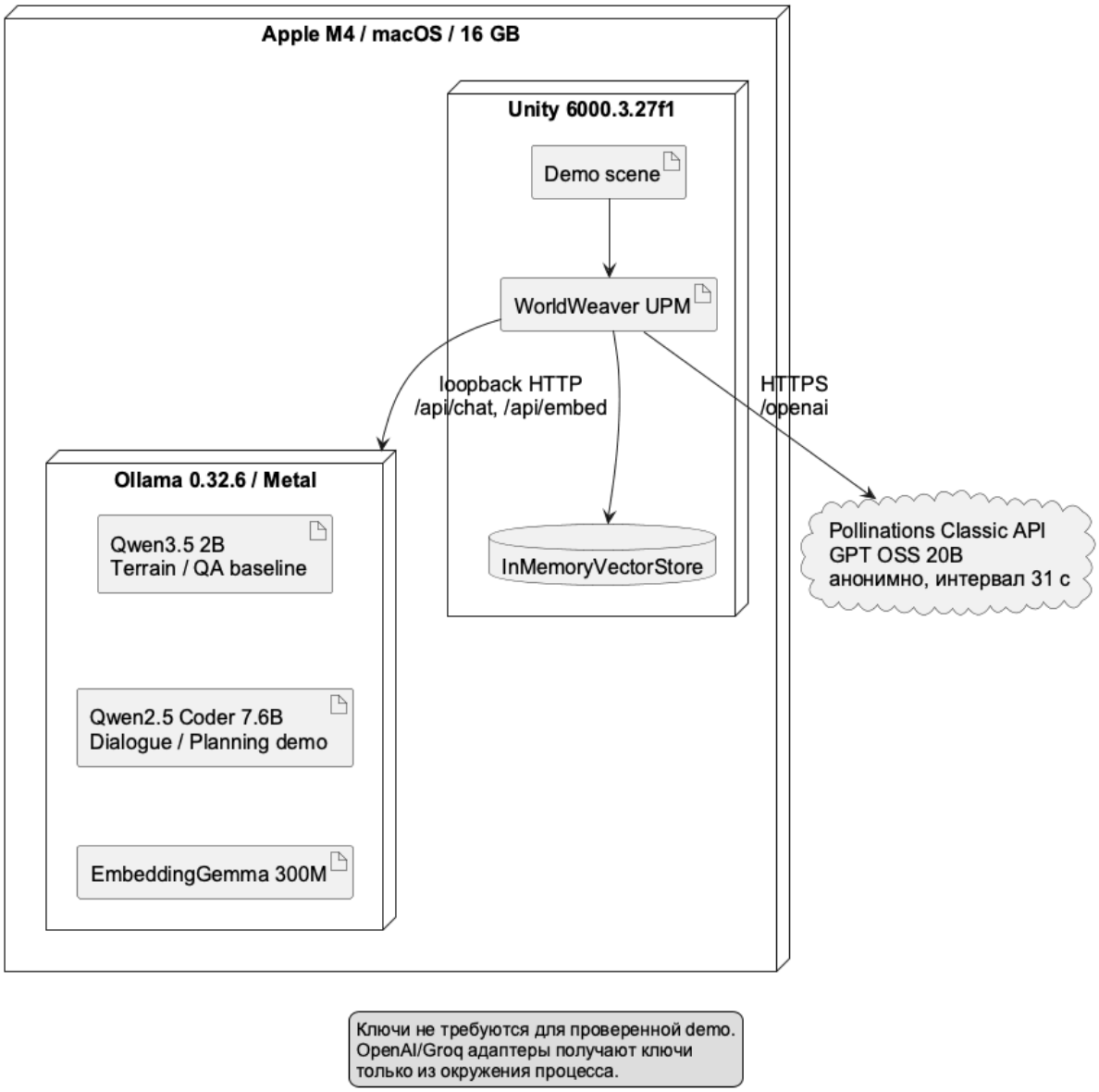


Рисунок 6 — Развёртывание локальных и облачных компонентов

Проектная архитектура обеспечивает наблюдаемую границу ответственности: языковая модель предлагает параметры и действия, retrieval выбирает доступные факты, а C# принимает решение о допустимости изменения состояния. Следующая глава описывает, как эти границы реализованы в программных компонентах.

3 РЕАЛИЗАЦИЯ И ДЕМОНСТРАЦИЯ

3.1 Среда разработки и состав поставки

Основная реализация выполнена на C# для Unity 6000.3.27f1. Эта LTS-версия установлена из официального архива после обнаружения требования Unity 6+ в ранее подготовленном техническом задании [1]. Изначально на компьютере присутствовала Unity 2022.3.62f3; в ней также проверены компиляция пакета и основной набор EditMode-тестов. Совместимость с дополнительными платформами не выводится из этих двух проверок.

Аппаратная среда включает Apple M4 и 16 ГБ объединённой памяти, операционная система — macOS 15.7.4. Локальный daemon Ollama имеет версию 0.32.6. Эти сведения нужны для интерпретации времени ответа: локальная производительность зависит от ресурсов, одновременно занятых редактором, моделей в памяти и прогрева inference.

Runtime-пакет разделён на каталоги Core, Providers, Orchestration, RAG, NPC, Actions, Dialogue и Terrain. В Editor находятся WorldWeaverWindow, TerrainAuthoring, DemoBuilder и средства воспроизводимого исследования. Исходники диаграмм, экспериментальные данные и текст работы хранятся рядом с кодом, но не входят в устанавливаемую runtime-сборку.

Для десериализации применяется официальный Unity-пакет Newtonsoft JSON. Он выбран из-за необходимости проверять произвольное JSON-дерево, типы токенов и повторение имён полей. Простое отображение JSON в сериализуемый объект не обеспечило бы всех проверок, поскольку неизвестные или ошибочно приведённые поля могли бы остаться незамеченными.

Ключи поставщиков не копируются в репозиторий. Файл .env.example перечисляет имена переменных без значений. Настоящий .env остаётся вне проекта, а launcher при необходимости передаёт окружение процессу Unity. Проверенная бесплатная демонстрация не требует ключей для локального Ollama и Pollinations.

3.2 Окно настройки в Unity Editor

WorldWeaverWindow открывается через Tools → Intelligent World Toolkit. Главное окно содержит пять разделов: WORLD, NPC, KNOWLEDGE, MODELS и DEBUG. Такая организация соответствует этапам работы разработчика: от создания геометрии к настройке персонажа, контекста и наблюдению результата.

Раздел WORLD содержит поле естественно-языкового описания, выбор маршрутов, запуск интерпретации и редактируемое представление TerrainSpec. Разработчик может сначала получить спецификацию от модели, затем проверить или исправить её и отдельно сформировать Terrain. Это разделение позволяет видеть параметры до изменения сцены и повторять генерацию без нового LLM-запроса.

При отсутствии обязательного модельного маршрута MODELS показывает предупреждение, позволяя завершить настройку профиля.

Кнопка Regenerate активна только для выделенного объекта WorldWeaver с маркером GeneratedTerrain. Благодаря этому произвольный Terrain стороннего проекта не становится целью автоматической перезаписи. Перед изменением TerrainData вызывается механизм Undo. Установка отменяемого действия не превращает всю файловую генерацию в транзакцию: новые ассеты могут сохраняться в Project после отмены объекта сцены.

Раздел NPC показывает выбранный IntelligentNPC и стандартные сериализуемые поля. Компонент задаёт имя, роль, личность, биографию, цели, мягкие ограничения, knowledge, routes, доступные сущности и разрешённые действия. Разработчик также определяет жёсткий лимит передачи и дальность взаимодействия. Эти поля представлены отдельно от свободного описания характера.

KNOWLEDGE связывает персонажа с KnowledgeBase, импортирует документы и запускает переиндексацию. При импорте извне Scope назначается текущему NPC, чтобы случайное содержимое JSON не изменило

область видимости. Размер файла ограничен одним мебибайтом. PDF не принимается и не выдаётся за успешно проиндексированный источник.

MODELS показывает отдельные назначения Terrain, Dialogue и Planning. Для модели отображаются публичный endpoint, model ID и только факт наличия требуемой переменной окружения. Health check проверяет доступность каталога и наличие модели; он не гарантирует, что будущая генерация будет разрешена балансом или текущим лимитом.

DEBUG представляет trace последнего pipeline. Здесь можно сопоставить исходный запрос, маршрут, найденные фрагменты, контекст, raw JSON, результат проверки и список исполненных действий. Таким образом, демонстрация допускает проверку каждого существенного шага, а не только финальной реплики.

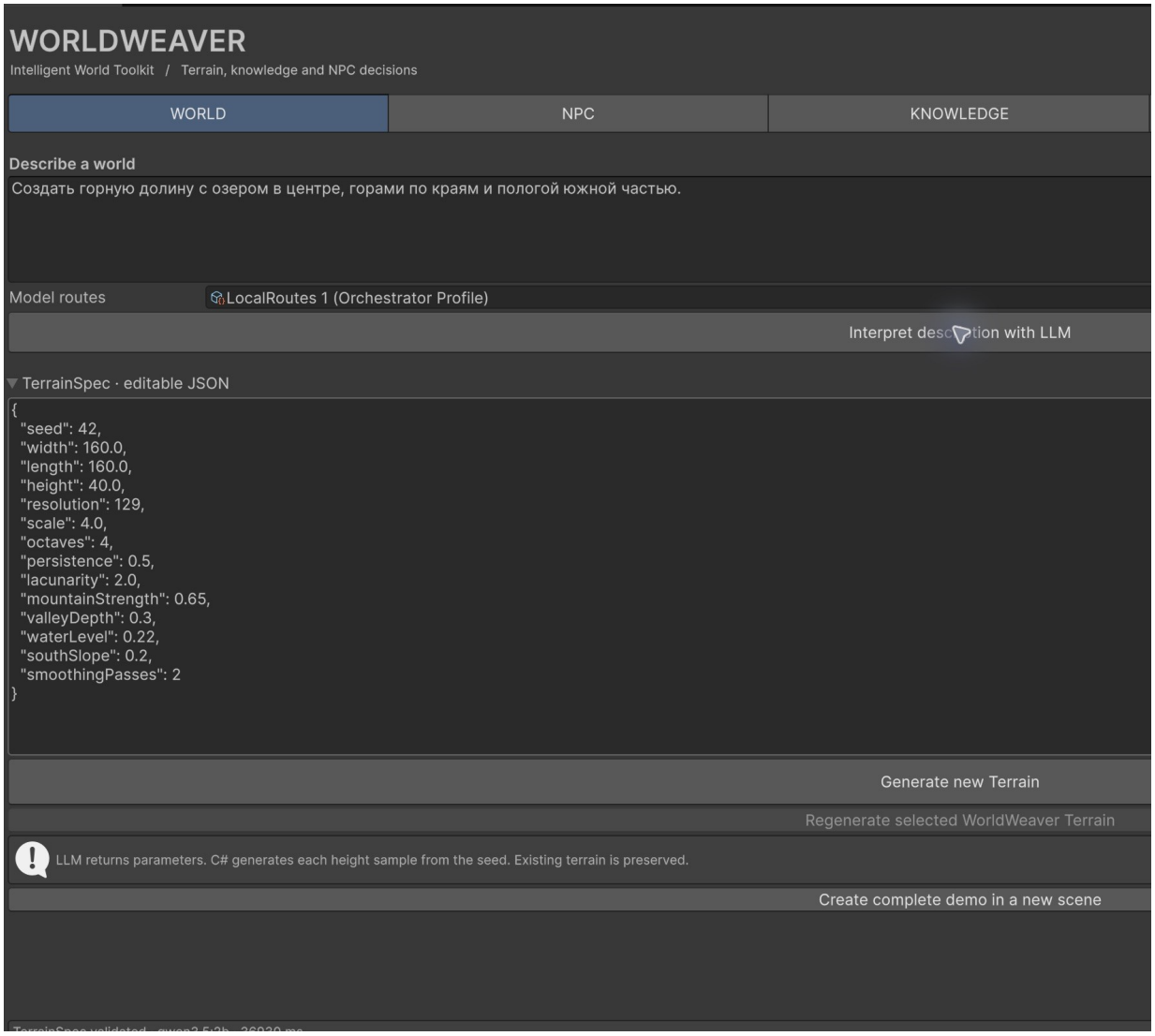


Рисунок 7 — Фрагмент раздела WORLD в Unity Editor

3.3 Процедурный генератор рельефа

TerrainGenerator не обращается к сети и не зависит от модели. На вход он получает проверенный TerrainSpec, на выход возвращает двумерный массив высот. Вычисление каждой точки включает сумму шумовых октав и геометрические преобразования поверхности. Все параметры ограничены до начала генерации.

В качестве основы применяется value noise с целочисленным смешиванием координат и seed. Интерполяция использует сглаженный коэффициент $t^2(3-2t)$, поэтому соседние клетки переходят друг в друга непрерывно. Для каждой октавы изменяются частота, амплитуда и seed, после чего сумма нормируется на суммарную амплитуду.

Горный край определяется нормированным расстоянием от центра. Центральная долина задаётся экспоненциальным понижением с различным масштабом по двум осям. Южное понижение изменяет вклад края в зависимости от координаты z. Эти операции дают контролируемое семейство ландшафтов, а не произвольную семантическую реконструкцию любого описания.

Сглаживание использует соседей по четырём направлениям. Новый массив создаётся на каждом проходе, поэтому ранняя запись одной ячейки не влияет на вычисление следующей в том же проходе. Для разрешения n и o октав вычислительная сложность основы составляет $O(n^2o)$, дополнительного сглаживания с p проходами — $O(n^2p)$.

TerrainAuthoring формирует TerrainData и назначает его созданному Terrain GameObject [3]. Дополнительная горизонтальная плоскость визуализирует воду. Три процедурно созданные одноцветные текстуры обозначают травянистую, каменную и снежную зоны в зависимости от высоты. Эти материалы помогают читать форму поверхности на защите, но не являются системой художественных биомов.

В текущей версии нет высотных пещер, симуляции гидрологии, ручного рисования зон или физической эрозии. Отсутствие этих функций не

мешает основному исследовательскому сценарию, поскольку проверяется переход от описания к контролируемым параметрам и повторяемому результату.

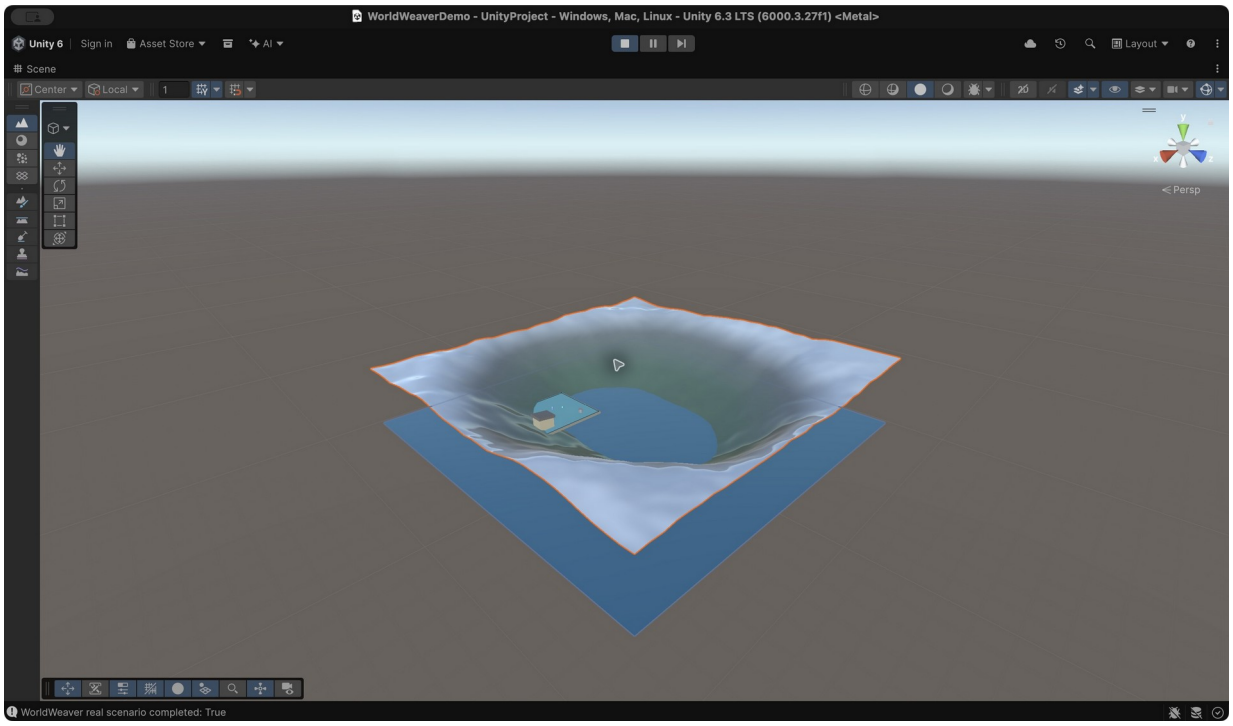


Рисунок 8 — Процедурный рельеф и демонстрационная деревня

3.4 Локальные и бесплатные облачные запросы

Для Terrain и сравнительного QA используется Qwen3.5 2B, а в итоговой демонстрации Dialogue и Planning — qwen2.5-coder:latest. Обе работают через Ollama на loopback-адресе; embeddings получает EmbeddingGemma. В локальном хранилище Qwen3.5 имеет 1,9 млрд параметров и квантование Q8_0, а qwen2.5-coder — 7,6 млрд и Q4_K_M. Различие моделей между QA и демонстрацией учитывается при выводах: качество диалога 7,6B не выводится из метрик 2B. Модели установлены в стандартном хранилище Ollama за пределами Git. При проблеме TLS штатного загрузчика использован HTTPS-загрузчик официальных manifests и blobs с обязательной проверкой SHA-256 каждого файла до регистрации manifest. Это обход технической ошибки загрузки, а не отключение проверки сертификатов.

Обычный запрос к local chat был проверен отдельно от Unity. Затем тот же provider вызывается из C#-pipeline. В запросе задаются schema,

ограничение числа выводимых токенов, контекст 4096 и отключение thinking для выбранной модели. Результат содержит обычный JSON-объект решения, который независимо разбирается приложением.

Первоначально предоставленный model ID openai/gpt-oss-120b проверялся через официальный Groq API: документация подтверждает такую модель в списке structured outputs, однако фактический endpoint из текущей сети возвратил HTTP 403 [22]. OpenAI позволил получить каталог моделей, но генерация вернула ошибку исчерпанного баланса. Доступность каталога оказалась недостаточным условием работоспособности inference.

По ограничению пользователя платные запросы исключены. Для проверенной cloud-конфигурации выбран Pollinations Classic API, который официально предоставляет анонимный OpenAI-compatible endpoint и GPT-OSS 20B [23]. Положительный ответ реального запроса сохранён отдельно. В runtime используется модельный псевдоним openai-fast, а trace сохраняет фактическое имя модели из ответа.

Classic API допускает один текстовый запрос за 30 секунд на IP. В provider установлен интервал 31 секунда и последовательный доступ через semaphore. Если сервер всё же возвращает ограничение с Retry-After, код ожидает разрешённый интервал в пределах настроенных повторов. Платёжные сообщения API не исполняются и не приводят к покупке токенов.

Для бесплатного API используется JSON Object Mode с текстовым описанием точной схемы. Это слабее строгого server-side schema-constrained decoding. Поэтому обе конфигурации проходят одинаковую C#-проверку NpcDecision, а доля валидных ответов измеряется отдельно. Различие режимов учитывается при интерпретации эксперимента.

Использование HTTP-провайдера не означает, что пакет включает production-сервер для коммерческой игры. В распространяемом клиенте нельзя надёжно скрыть ключ облачного аккаунта. Для настоящего многопользовательского продукта понадобится доверенный backend с

собственной авторизацией, квотами и наблюдением. Текущая бесплатная demo работает без сохранённого API-ключа.

3.5 Реализация retrieval и индексации

Статические сведения демонстрации описывают деревню Лунный Брод, торговца Мирона, лечебный кристалл, лекаря, дом, колодец, праздник, мост, шахту и архив. Эти сведения являются авторскими тестовыми данными. Их нельзя считать реальными историческими или медицинскими фактами; они специально созданы для вопросов с известными ответами.

Дополнительно создан документ стража с отдельным Scope. Он содержит служебный пароль и используется для проверки изоляции. Торговец не получает этот документ в свой индекс. Проверка не ограничивается надеждой на то, что модель проигнорирует чужой текст: отбор выполняется до обращения к генератору.

При первом запросе NpcPipeline вычисляет fingerprint состава KnowledgeBase и при необходимости запускает ReindexAsync. Векторы для chunks получает OllamaEmbeddingProvider. После успешной индексации каждый вопрос преобразуется в query-вектор, а InMemoryVectorStore возвращает ранжированные результаты. В debug сохраняются DocumentId и score, что позволяет проверить происхождение контекста.

Разделение типов памяти позволяет не смешивать установленные факты и недавние события. Статическая база отражает авторский лор; разговорная память содержит реплики; эпизодическая память фиксирует исполненные действия. Ограниченная история не отправляется целиком. Из каждой группы выбираются наиболее близкие фрагменты.

В сравнительном QA-эксперименте накопление разговорной памяти выключено. Иначе ранние ответы могли бы становиться дополнительными знаниями для следующих вопросов, а различия между конфигурациями отражали бы не только статический RAG. Для живой демонстрации память включена и обновляется после обработки диалога.



Рисунок 9 — Просмотр базы знаний отдельного NPC

Переиндексация и запрос используют разные жизненные циклы. Перестроение требуется после изменения текста или scope; обычная реплика получает только один query-вектор и выполняет поиск по готовому индексу. В текущей версии fingerprint вычисляется из сериализованного набора документов персонажа. Это обеспечивает обнаружение изменений в Editor,

но на большом корпусе само вычисление fingerprint следует заменить версией ассета или журналом изменений.

При импорте Markdown разметка хранится как текст. Не выполняется загрузка внешних ссылок и не запускается код из документа. JSON-источник принимается в предусмотренной структуре документов, после чего область видимости назначается активному NPC. Эти правила ограничивают неожиданное расширение доступа при переносе базы из чужого проекта.

3.6 Обработка диалога и структурированного ответа

Метод IntelligentNPC.TalkAsync проверяет отсутствие уже активного запроса, переводит NPC в Thinking и создаёт связанный cancellation token. NpcPipeline получает данные мира, выполняет retrieval и формирует CompletionRequest. System-сообщение задаёт правила использования роли, знаний и разрешённых действий, а пользовательская часть содержит JSON с контекстом и исходной репликой.

Контекст включает инвентарь, состояние NPC, расстояния до известных сущностей, цель, personality и hard_constraints. Число сущностей ограничено 32, дополнительных адаптеров мира — восемью. IWorldContextSource позволяет игровому проекту добавить собственное компактное состояние без изменения provider или router.

После получения CompletionResult вызывается NpcDecision.Parse. Строго проверяются обязательные имена и типы, а повторённое поле считается ошибкой. Markdown-обёртка не извлекается эвристически: если модель вернула текст вне контракта, pipeline фиксирует нарушение. Такое решение облегчает диагностику и не выдаёт частично угаданный объект за правильный ответ.

Корректно разобранное решение передаётся в ActionDispatcher. Принятый результат позволяет показать speech. При отклонении действия интерфейс сообщает причину; обещание модели не отображается как свершившееся изменение инвентаря. Trace содержит raw output и

фактический результат проверки, поэтому несоответствие остаётся наблюдаемым.

Отдельный компонент DemoDialogue предоставляет поле ввода, отправку, кнопку автономного решения и переключение local/cloud. Рядом показаны реальные остатки Crystal у NPC и Player. Эти значения читаются из Inventory после исполнения и не являются извлечёнными из речи числами.

Жизненный цикл памяти отделён от уже исполненного действия. Если после commit передача завершена, а обновление embedding-памяти отменено, trace сохраняет исходный результат и предупреждение об индексации. Пользователь получает подтверждённое состояние сцены. Возврат общей ошибки Cancelled после передачи мог бы создать ложное впечатление, что операция не состоялась, и спровоцировать повтор. Для этого случая добавлена отдельная регрессионная проверка с фактическими остатками 9/1.

3.7 Передача предмета и автономное движение

GiveItem является встроенным примером действия с последствиями. Для него проверяются разрешение, зарегистрированная цель, дальность, положительное количество, остаток и вместимость получателя. Общий бюджет решения не позволяет обойти лимит несколькими небольшими командами. В демонстрации у торговца изначально десять кристаллов, а лимит передачи за решение равен одному.

TakeItem представлен тем же TransferAction с обратным направлением, но требует дополнительного AllowTakingItems. Наличие такого обработчика в реестре само по себе не даёт NPC права использовать действие. В стандартной demo оно не входит в список разрешённых операций торговца.

MoveTo, Follow и Flee используют NavMeshAgent. Валидатор сначала проверяет положение агента на NavMesh, затем ищет допустимую конечную точку и полный путь. Для Flee конечная точка вычисляется детерминированно в направлении от угрозы; модель выбирает сам тип действия и цель угрозы, а не промежуточные координаты движения.

Follow обновляет destination с интервалом 0,5 секунды. Для этого не выполняется новый LLM-запрос. Планирование NPC ограничено PlanningCooldown, по умолчанию 45 секунд, и не запускается параллельно активному диалогу. Автономный режим можно включить в Inspector; для демонстрации предусмотрен также явный запрос следующего решения.

DemoNavigation строит небольшой NavMesh только по назначенной walkable-геометрии рыночной площади [4]. Это позволяет проверить настоящее движение и отделить его от художественного качества рельефа. Проект не объявляет, что автоматически построил навигацию по всем сгенерированным горам и водной поверхности.

Состояния Idle, Thinking, Moving, Following, Waiting и Failed отражают основные этапы работы. Ошибка API переводит NPC в Failed, но не разрушает GameObject и не блокирует новую попытку. Отмена завершает активную операцию и возвращает управление. После достижения MoveTo-цели агент переходит в Idle.

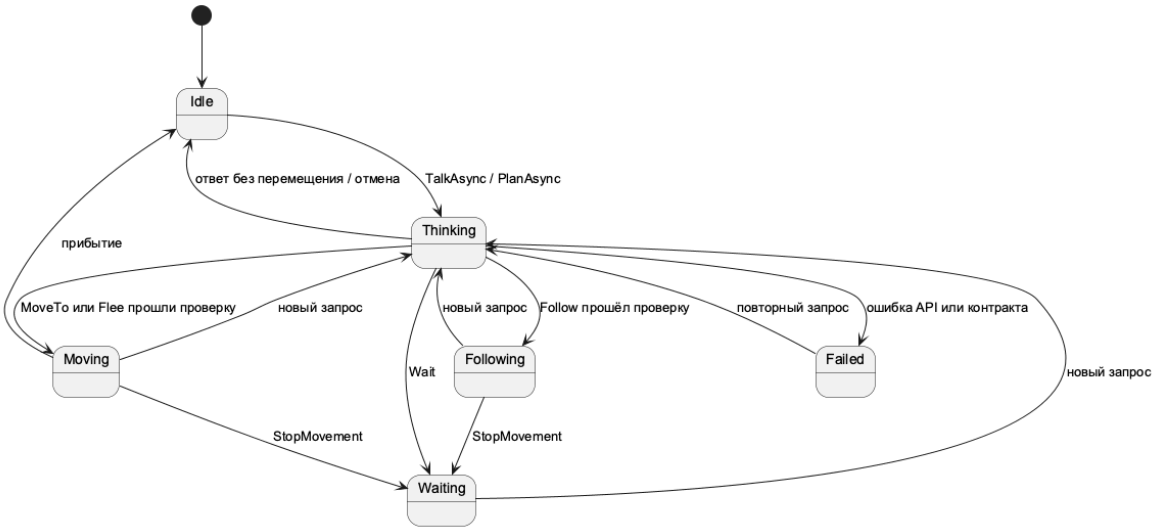


Рисунок 10 — Состояния интеллектуального персонажа

Для автономного Planning предусмотрено отдельное PlanningActions. Итоговый список является пересечением с AllowedActions, поэтому расширение общих возможностей NPC не открывает автоматическую передачу предмета во время периодического планирования. Этот список также включается в enum схемы NpcDecision. Повторная проверка dispatcher обеспечивает правило даже при игнорировании схемы внешним API.

Разговорная память исключается из контекста Planning, поскольку прежняя просьба игрока могла ошибочно восприниматься как новое действие. Эпизодическая запись сохраняется и явно обозначает уже исполненное. Контекст сообщает позицию, расстояния и текущую цель. Такой подход не превращает модель в детерминированный планировщик, но устраняет обнаруженный путь повторного исполнения диалогового намерения.

3.8 Диагностика и ограничения безопасности

PipelineTrace фиксирует время, персонажа, input, route, retrieved chunks, ограниченный контекст, raw JSON, provider/model, токены, результат валидации и список действий. Журнал в памяти ограничен 30 записями. Экспорт JSON предназначен для технической проверки и исследования, а не для хранения ключей.

Код не журналирует заголовок Authorization и не сохраняет сырые тела provider-ошибок, поскольку такие сообщения иногда включают часть credential. В профиле запрещены credentials внутри URL. Для локального API разрешён HTTP только на loopback, а удалённый endpoint должен использовать HTTPS.

Эти меры не делают вывод модели доверенным. В контекст могут попасть атакующие инструкции из реплики или документа. Prompt отделяет такие данные от правил, но окончательный барьер для действий — список разрешений и проверка текущего состояния. При этом система не гарантирует отсутствие любых нежелательных реплик: контентная политика диалога и формальная допустимость действия являются разными задачами.

Набор жёстких проверок конечен. Текстовые Constraints не преобразуются автоматически в доказуемые правила. Если игровому проекту требуется, например, запрещать торговлю ночью или учитывать принадлежность предмета, разработчик должен добавить соответствующее

детерминированное условие или собственный обработчик. Это ограничение отражает область применимости инструментария.

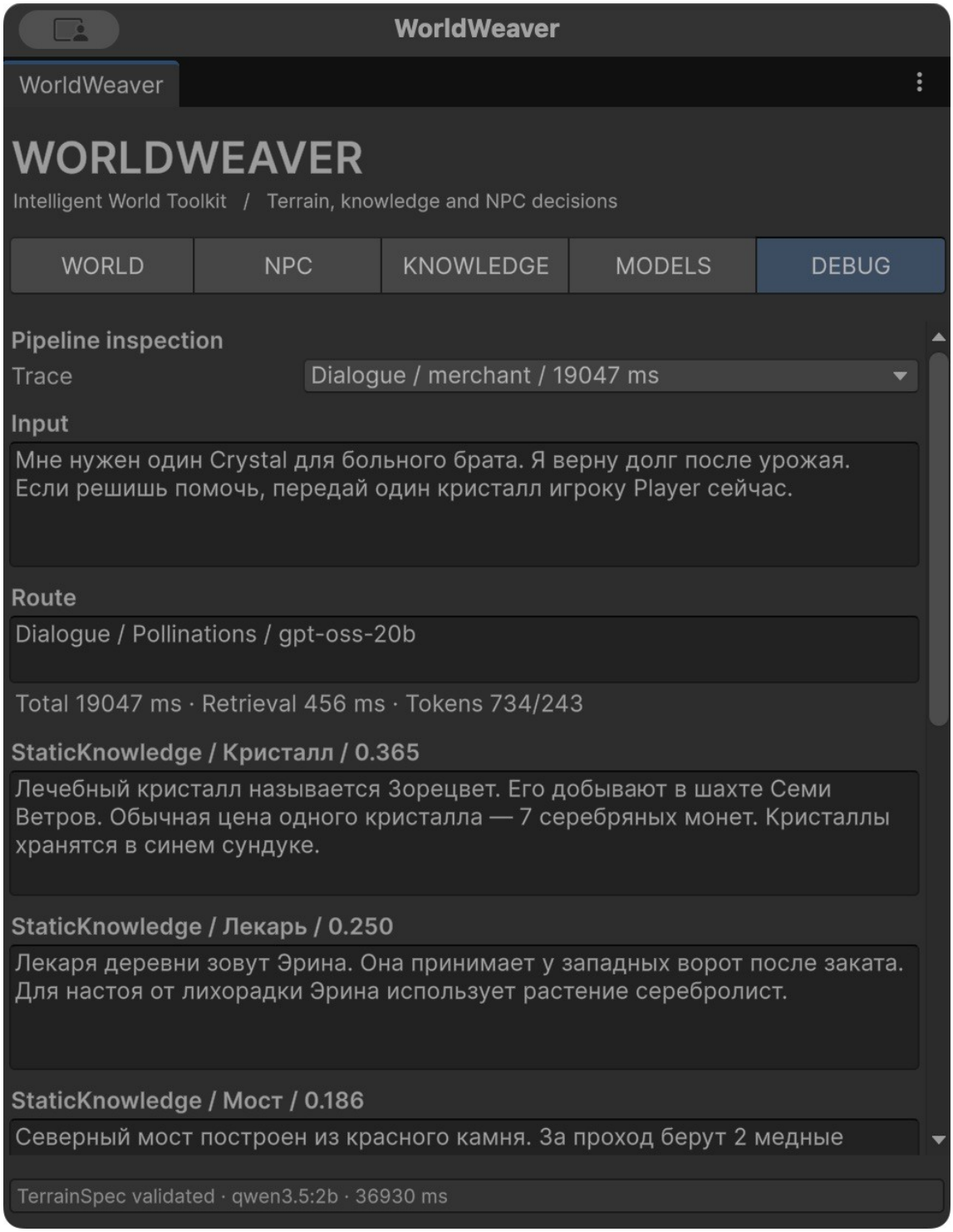


Рисунок 11 — Диагностика маршрута и извлечённых знаний

3.9 Демонстрационный сценарий

DemoBuilder сохраняет новую сцену по уникальному пути и добавляет её к существующему списку сборки. При интерактивном запуске предварительно предлагается сохранить изменённую сцену. Демонстрация

содержит Terrain, водную зону, рыночную площадь, Player, торговца, стража, дом и колодец. Геометрия намеренно проста: она нужна для различения целей и движения, а не для имитации готовой коммерческой игры. На торговце размещаются IntelligentNPC, Inventory, WorldEntity и NavMeshAgent.

Последовательность защиты начинается с подключения пакета и открытия окна. Разработчик вводит описание рельефа, получает TerrainSpec и генерирует поверхность. Затем он выбирает торговца, показывает личность, базу знаний и маршруты. В Play Mode игрок спрашивает о факте лора, после чего предлагает помощь больному брату и просит один кристалл.

При согласии модели JSON содержит GiveItem, validator проверяет условия, а Inventory показывает изменение остатка. Следующий Planning-запрос предлагает NPC выбрать действие для своей цели. Если выбрано MoveTo к House, NavMeshAgent физически перемещает объект. В Debug можно сопоставить route, ответ, проверки и выполненную команду.

Смена local/cloud повторяет разговор через другой provider без изменения игрового кода. Для бесплатного cloud заранее учитывается 30-секундный интервал. Если сервис недоступен, демонстрация показывает контролируемую ошибку и остаётся работоспособной в локальном режиме. Успешный end-to-end результат подтверждается отдельным JSON-протоколом, а не наличием mock-теста.

Реализованный продукт объединяет основные процессы в одном пакете и сохраняет расширяемость на уровне интерфейсов. Его фактическое качество оценивается в следующей главе через автоматические проверки, известные ответы и измеренные задержки.

4 ТЕСТИРОВАНИЕ И ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА

4.1 Цели и организация проверки

Испытания разделены на проверки программных инвариантов, интеграцию с реальными моделями и эксперимент над ответами NPC. Эти группы подтверждают разные свойства. Mock-провайдер позволяет воспроизвести точный ошибочный ответ и проверить отсутствие побочного эффекта. Реальный provider подтверждает сетевую интеграцию, но его единичный успешный ответ не доказывает устойчивость на всех репликах.

Программные испытания выполнены средствами Unity Test Framework [24]. EditMode проверяет чистые алгоритмы, конфигурацию, разбор JSON, retrieval, память и HTTP. PlayMode проверяет изменение Inventory и движение NavMeshAgent. Отдельный сценарий RealScenario вызывает настоящую модель, получает структурированное решение и сохраняет реальные изменения сцены.

Для QA-эксперимента подготовлено 30 фиксированных вопросов с известными ответами. Три конфигурации образуют 90 запросов: получено 89 ответов модели и одна ошибка HTTP 400; ошибочный запрос сохраняется в знаменателе метрик качества. Лор является авторским и содержит конкретные имена, цены, места и правила вымышленной деревни. Такой набор позволяет отделить сведения, доступные из базы, от общих знаний модели. Для каждого вопроса сохранён идентификатор правильного документа и допустимые словесные варианты ответа.

Исследуются три конфигурации: локальная Qwen3.5 2B без RAG, та же модель со статическим RAG и облачная GPT-OSS 20B через бесплатный API с тем же RAG. Embedding-модель и алгоритм retrieval сохраняются одинаковыми. Смешанная оркестрация дополнительно проверяется в демонстрационном сценарии через отдельные Terrain, Dialogue и Planning routes.

Файл сценариев и построчные JSON-протоколы находятся в results. Каждая строка содержит исходный вопрос, конфигурацию, ожидаемые варианты, trace или контролируемую ошибку, время и результат автоматической оценки. Агрегаты строятся из этих файлов; итоговые числа не задаются вручную в графиках.

4.2 Модульные и сетевые испытания

Основной EditMode-набор итоговой версии включает 56 проверок. Для Terrain проверяются допустимость разрешения и числовых значений, обязательные поля, отклонение дробного integer и неизвестного поля, одинаковый seed и нормированный диапазон высот. Сравнение двух массивов в одной среде выполняется непосредственно по значениям, без визуального предположения о сходстве.

Отдельная регрессионная проверка отменяет индексацию памяти после уже принятого GiveItem: речь и результат исполнения должны сохраниться, а оба остатка составить 9 и 1. Это проверяет границу commit, где отмена вспомогательной операции не должна маскировать состоявшееся изменение мира.

Для Inventory и действий проверяются изменение обоих участников передачи, неправильное количество, неизвестные item и target, запрет операции, дальность, вместимость, повторение идентификаторов сущностей и общий бюджет. Отдельный сценарий проверяет двойное расходование: два действия запрашивают больше доступного количества в сумме. Предварительная проверка должна отклонить весь пакет без частичной передачи.

Разбор NpcDecision проверяется на недостающих и лишних полях, повторённом speech, строковом amount и Markdown-обёртке. Эти случаи важны для работы с model output: синтаксически похожий ответ не должен попадать в игровой state через неявное преобразование.

RAG-проверки включают перекрытие chunk-окон, устойчивость идентификатора, изменение scope, ранжирование близких векторов, исключение документа другого персонажа и несовместимость размерностей. Ограничение памяти проверяется после добавления большего числа записей, чем разрешено. Отмена переиндексации не должна уничтожать ранее построенный индекс.

Сетевые проверки используют локальный TCP-сервер с заранее заданными ответами. Сервер возвращает 429 с последующим 200, ошибку 400, неправильный JSON либо задерживает ответ. Код проверяет повтор временной ошибки, отсутствие повторов для клиентской ошибки, cancellation и timeout. Использование такого сервера позволяет испытать сбои без намеренного расходования квоты облачного API.

Таблица 5 — Модульные и сетевые испытания

Набор	Результат	Ошибки	Длительность или условие
Unity 6 EditMode	56/56	0	2,32 с
Unity 6 PlayMode без внешних моделей	2/2	0	1,08 с
Unity 6 PlayMode с реальными моделями	2/2	0	85,16 с
Unity 2022.3 EditMode, исходная редакция	54/54	0	2,34 с
Чистая установка local-tarball и Git	Успешно	0 ошибок компиляции	Два новых проекта и новые Library

Результаты тестов в таблице получены из XML-протоколов Unity, результат установки — из сохранённого JSON-отчёта. Наличие положительного mock pipeline не считается доказательством внешнего inference. В нём проверяется только соединение retrieval, парсера и dispatcher при управляемом ответе.

4.3 Интеграция в Play Mode

Передача Crystal проверяется по обоим инвентарям. Сценарий начинается с десяти предметов у NPC и нулевого количества у Player. После принятого GiveItem(1) ожидаются девять и один соответственно. При отклонённом решении оба остатка должны сохраняться.

Проверка движения строит NavMesh по назначенному walkable-мешу, размещает агент на сетке и отправляет MoveTo. После заданного интервала измеряется фактическое изменение позиции. Проверка Follow дополнительно подтверждает переход в соответствующее состояние и остановку через StopMovement. Это проверка детерминированного исполнительного слоя; необходимость LLM для неё отсутствует.

Реальный сценарий расширяет эту проверку запросом к модели. Сначала NPC получает просьбу помочь больному брату и передать один Crystal. Затем отдельный Planning-запрос предлагает выбрать действие для текущей цели. Протокол содержит исходный и конечный остатки, raw JSON, результат валидации и пройденное расстояние.

Таблица 6 — Интеграция в Play Mode

Маршрут	Ходов диалога	Crystal NPC / Player	Изменение позиции	Итог
Полностью локальный	2	9 / 1	11,6803 м	Пройден
Смешанный с бесплатным облачным Dialogue	1	9 / 1	11,6437 м	Пройден

Для каждого маршрута выполнен один финальный ограниченный переговорный сценарий. Общий протокол содержит до трёх заранее определённых реплик: просьбу помочь, согласие на цену в семь монет с обязательством возврата долга, затем подтверждение условий. После появления одного Crystal у игрока цикл прекращается, затем вызывается Planning. В облачном варианте передача произошла на первом ходе, в локальном — на втором. В обоих случаях Planning выполнила локальная qwen2.5-coder:latest и предложила MoveTo(House). Следовательно, это

проверка завершения двух конкретных сценариев, а не оценка общей вероятности успешных переговоров.

Локальный финальный ответ выявил ограничение: при принятом GiveItem речь сохранила осторожную формулировку о необходимости сначала убедиться в возврате. Изменение обоих инвентарей действительно произошло, но смысловое согласование речи и поступка оказалось неполным. Интерфейс поэтому отображает остатки из Inventory. Структурная валидность и корректность исполнения не гарантируют художественную последовательность текста.

Дополнительный повтор через графический Editor подтвердил ограниченную устойчивость поведения. В облачном GUI-сценарии передача и перемещение вновь состоялись. В локальном GUI-сценарии после двух реплик Inventory также изменился до 9/1, но последующее Planning вернуло пустой actions и речь с отказом; изменение позиции составило 0 м. JSON прошёл проверку, поскольку бездействие допустимо контрактом, однако полный демонстрационный критерий не был выполнен. Эти данные сохранены в gui-scenario-local.json и gui-scenario-cloud.json. Поэтому положительный результат двух интеграционных тестов не интерпретируется как гарантированное планирование при любом повторении.

Ранние прогоны сохранены в каталоге iterations. В них встречались допустимый отказ от подарка, выбор Wait и повторение передачи в Planning. Инженерное исправление ограничило Planning собственным подмножеством действий и enum в JSON Schema; разговорные реплики исключены из planning-контекста, а эпизоды описывают уже исполненное. Это уменьшает возможность повторить просьбу из прошлой реплики. Проверка разрешений в C# остаётся обязательной и после усиления схемы.

Проверка поставки выполнена двумя способами: из local-tarball и из Git-зависимости с закреплённым commit. В обоих случаях создан новый проект с новой Library; число ошибок компиляции равно нулю. Дополнительный consumer-сценарий дважды создал demo, сохранил

исходную сцену и порядок существовавших build settings, подтвердил двух NPC, ссылки KnowledgeBase и маршрутов. Изменение параметров Terrain с последующим Regenerate изменило также alphamap трёх слоёв. Эти результаты сохранены отдельно от модельного inference.

Финальная интеграция выполнена средствами Unity Test Framework: два теста, оба Passed. Чистая установка отдельно подтвердила импорт архива пакета в новый проект и создание демонстрационной сцены без ошибок компиляции. Она не заменяет модельные испытания и не измеряет доступность стороннего API.

Наблюдаемое движение подтверждает исполнение конкретного действия на демонстрационной площади. Оно не доказывает автоматическую проходимость любого сгенерированного Terrain. Если модель выбирает допустимый Wait вместо MoveTo, это валидное решение pipeline, но оно не удовлетворяет отдельному демонстрационному критерию физического перемещения; эти два результата не смешиваются.



Рисунок 12 — Диалог в Play Mode и состояние инвентарей

4.4 Метрики качества ответов и retrieval

Основная метрика QA — доля вопросов, в ответе на которые присутствует допустимая нормализованная формулировка известного факта. Для текста приводятся регистр и буква ё, для числовых вариантов проверяются границы цифровой последовательности. Это автоматический proxy корректности факта, а не полная экспертная оценка смысловой достоверности.

Такой критерий может пропустить противоречие: модель способна назвать ожидаемое имя и одновременно добавить неверное объяснение.

Поэтому результаты нельзя называть абсолютной groundedness. Сырые реплики сохраняются для последующей ручной проверки, а в описании метрики используется формулировка «совпадение с известным ответом».

Retrieval hit@3 показывает долю вопросов, для которых правильный документ присутствует среди трёх статических фрагментов. Дополнительно рассчитывается reciprocal rank первого правильного источника. Precision@3 интерпретируется с учётом разметки: для каждого вопроса задан один целевой документ, поэтому наличие одного нужного фрагмента среди трёх соответствует 1/3, а не полному заполнению выдачи релевантными объектами.

Доля JSON-valid отражает успешность строгого прикладного парсера. Результат ActionDispatcher характеризует принятие предложенного пакета действий. Для вопросов о фактах большинство корректных ответов не требует предметных изменений, поэтому высокий процент принятых пустых actions или Talk не доказывает качество сложного планирования. Надёжность предметных операций оценивается отдельными тестами.

Для каждой конфигурации измеряются полное время запроса, время retrieval и время провайдера, а также доступные счётчики input/output tokens. Полное время включает ожидание квоты, подготовку и первичную индексацию, если она потребовалась. Время провайдера отражает более узкий участок сетевого обращения и не равно чистому времени вычислений модели.

Таблица 7 — Метрики качества ответов и retrieval

Показатель	**Qwen3.5 2B без RAG**	**Qwen3.5 2B с RAG**	**GPT-OSS 20B с RAG**
Факт в исходной речи	2/30	20/30	29/30
Факт в принятой реплике	1/30	16/30	16/30
JSON прошёл парсер	30/30	30/30	29/30
Пакет действий принят	27/30	22/30	16/30
Hit@3	0/30	30/30	30/30

Показатель	Qwen3.5 2B без RAG	Qwen3.5 2B с RAG	GPT-OSS 20B с RAG
MRR@3	0,000	0,983	0,983
Ошибка API	0	0	1

Без RAG локальная модель назвала ожидаемый факт в 2 из 30 исходных реплик. При добавлении одинаковой базы знаний результат составил 20 из 30, то есть прирост на 60 процентных пунктов в этом наборе. Все 30 целевых документов вошли в top 3, а MRR@3 составил 0,983. При этом модель не использовала найденный факт правильно в десяти вопросах. Таким образом, безошибочный hit@3 не означает безошибочную генерацию.

Облачная конфигурация содержала ожидаемый факт в 29 из 30 исходных реплик. На вопросе qa-26 API возвратил HTTP 400; строка сохранена как ошибка и остаётся в знаменателе. Значение 29/30 не следует интерпретировать как отсутствие иных неточностей в этих репликах: метрика проверяет наличие размеченного факта, а не все дополнительные утверждения.

Прикладной результат ниже исходного текста модели. До игрока дошёл ожидаемый факт в 16 из 30 случаев как для локального RAG, так и для облачного. В облачной группе 13 содержательно подходящих ответов были отклонены из-за Talk с target Player, тогда как действовавший контракт требовал пустой target. В локальных группах встречались также перемещения в Editor вне NavMesh. Эти случаи показывают чувствительность конечного результата к договорённости о форме действия и к условиям сцены.

Эксперимент выполнен в Editor, а не в полноценном игровом окружении каждого вопроса. Поэтому доля принятых пакетов не является самостоятельной оценкой навигации. Исполнительный слой обоснованно отклонял MoveTo без активной NavMesh. Отдельные PlayMode-тесты проверяют перемещение в предназначенной для него среде.

После эксперимента уточнены подсказки и схема допустимых действий для Planning. Сохранённые 90 запросов не пересчитывались как результаты исправленной модели. Для доказательства улучшения конечной QA-точности нужен новый прогон с тем же протоколом и новым обозначением версии. Результат работы включает как достигнутые показатели, так и воспроизводимо выявленную проблему согласования семантики model output с контрактом действий.

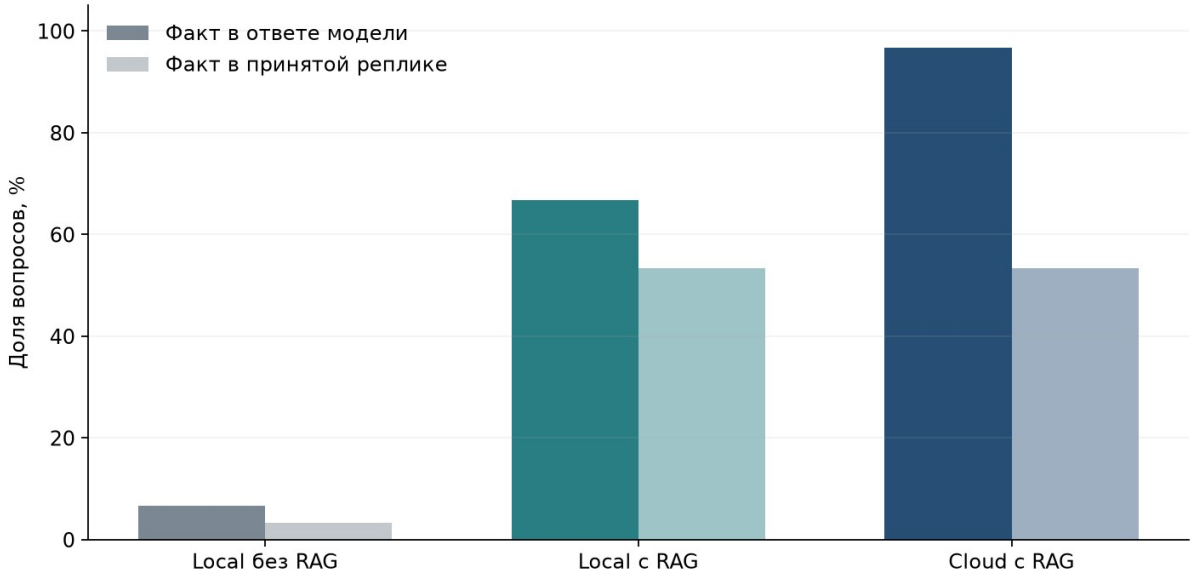


Рисунок 13 — Доля совпадений факта в исходной и принятой реплике

4.5 Локальное и облачное исполнение

Одинаковый прикладной контракт позволяет менять provider, но не делает условия исполнения равными. Локальная модель использует ресурсы компьютера и может конкурировать с Unity за память. Облачная модель имеет другие параметры и серверное оборудование, а бесплатный маршрут дополнительно ограничивает частоту. Поэтому разность времени не следует приписывать только размеру LLM.

Таблица 8 — Локальное и облачное исполнение

Конфигурация	p50 полного запроса, с	p95 полного запроса, с	p50 обращения provider, с
Qwen3.5 2B без RAG	6,33	7,33	6,32
Qwen3.5 2B с RAG	6,95	7,92	6,60
GPT-OSS 20B с RAG	30,95	74,54	10,20

Медиана времени provider рассчитывается по строкам с доступным ненулевым ElapsedMs. Отсутствие этой метрики при ошибке API не интерпретируется как нулевая задержка.

Для Pollinations предусмотрен последовательный доступ с интервалом 31 секунда. При оценке пользовательского ожидания этот интервал включается в полную задержку. Исключать его из общего времени означало бы скрыть существенное ограничение бесплатного варианта. Одновременно отдельно приводится время конкретного HTTP-обращения, чтобы различать ожидание слота и получение completion.

Сравнение не требует покупки токенов. Использованные локальные запросы выполняются на оборудовании пользователя, а анонимный cloud-маршрут предоставлялся бесплатно на дату проверки. Нулевая стоимость API-запроса не означает отсутствия затрат электроэнергии, оборудования, разработки или сопровождения. Экономия трудоёмкости относительно ручной настройки не измерялась контролируемым исследованием, поэтому процент сокращения времени не заявляется.

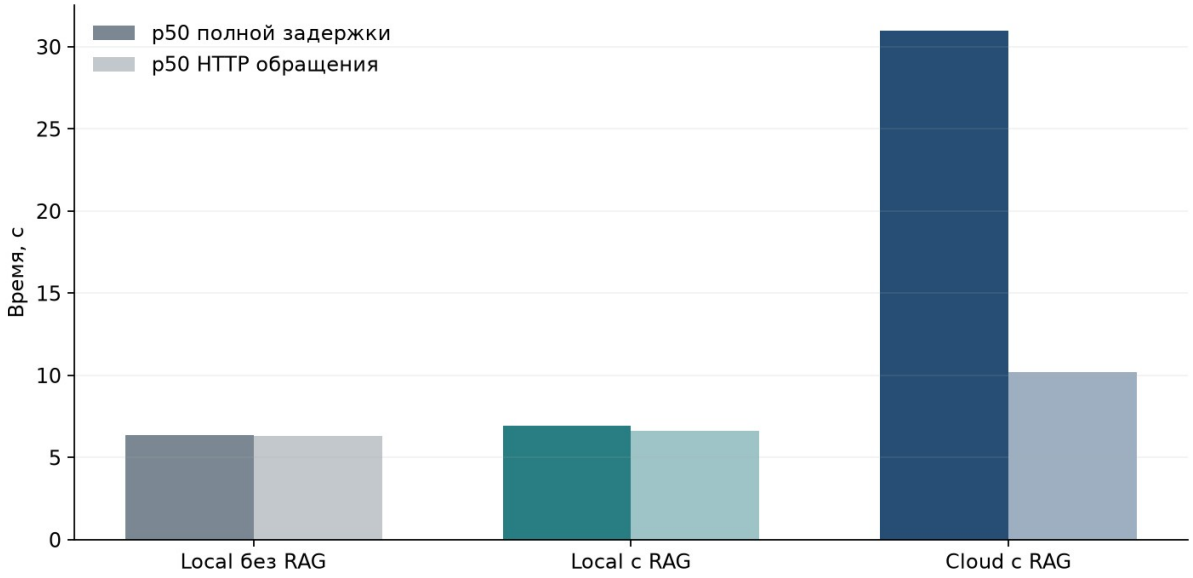


Рисунок 14 — Измеренные задержки модельных конфигураций

4.6 Производительность и воспроизводимость Terrain

Для Terrain выполнена серия из десяти seed на каждом из четырёх разрешений: 65, 129, 257 и 513. Измеряется непосредственно вызов TerrainGenerator.Generate в C#. Сохранение ассетов, работа интерфейса, получение TerrainSpec от LLM и импорт материалов в это время не входят.

Каждая спецификация генерируется дважды, и массивы сравниваются поэлементно. Это позволяет проверить воспроизводимость в конкретной версии Unity и на конкретной машине. Совпадение между разными платформами, компиляторами и режимами вычислений отдельно не исследовалось и не объявляется доказанным.

Таблица 9 — Производительность и воспроизводимость Terrain

Разрешение	**Число seed**	**Медиана Generate, мс**	**p95 Generate, мс**
65	10	2,237	3,846
129	10	8,718	8,756
257	10	34,223	36,445
513	10	136,130	137,208

Рост времени с разрешением согласуется с увеличением числа ячеек. Увеличение стороны сетки примерно вдвое увеличивает их число примерно

вчетверо, поэтому сравнивать результаты только по размеру n без учёта n^2 некорректно. Для интерфейса выбран умеренный размер по умолчанию, который сохраняет читаемость формы и небольшой объём данных.

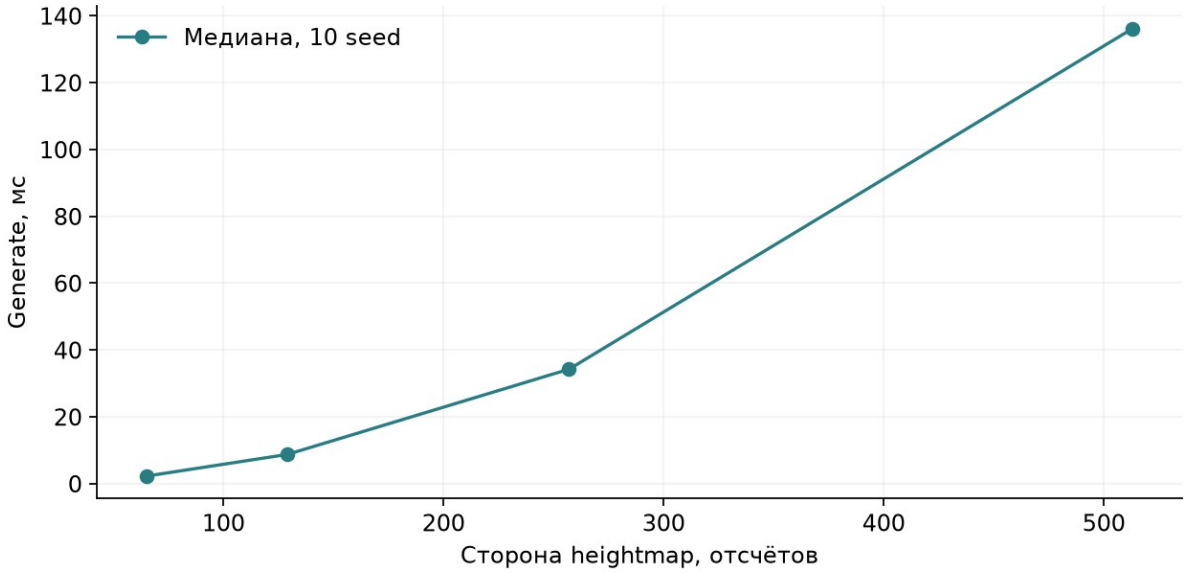


Рисунок 15 — Продолжительность алгоритмической генерации Terrain

4.7 Формализация метрик и проверка воспроизводимости

Для каждого вопроса i задаётся индикатор f_i, равный единице при наличии хотя бы одного допустимого варианта известного ответа в сгенерированной речи. Доля совпадений составляет $F = (\Sigma\ f_i)/n$. Второй индикатор g_i дополнительно требует, чтобы решение прошло прикладной валидатор и соответствующая речь была выдана игроку. Следовательно, $G = (\Sigma\ g_i)/n$ характеризует конечный пользовательский результат. Условие $g_i \le f_i$ для каждого случая помогает обнаружить ошибочное смешение этих показателей.

Для retrieval индикатор h_i равен единице, если ожидаемый DocumentId найден в первых трёх статических результатах. $Hit@3 = (\Sigma\ h_i)/n$. Если документ находится на позиции r_i, его reciprocal rank равен $1/r_i$; при отсутствии правильного документа принимается ноль. MRR@3 получается усреднением этих величин. В наборе с одним размеченным документом на вопрос эти метрики интерпретируются проще, чем precision, и не требуют считать все остальные фрагменты заведомо нерелевантными.

Для задержки используется медиана и 95-й перцентиль. Отсортированные времена обозначаются $t_0,\ldots,t_{n-1}$. Для доли p вычисляется индекс $j = (n-1)p$ и линейная интерполяция между соседними значениями. Эта конвенция зафиксирована в скрипте анализа, поскольку различные библиотеки могут выдавать разные перцентили на малой выборке. Время ошибки также сохраняется: исключение не удаляет неудобный случай из знаменателя качества.

Доверительные интервалы для доли совпадений рассчитаны методом Уилсона при $z = 1{,}96$. Они показывают статистическую неопределённость при условной модели независимых испытаний и не устраняют зависимость вопросов от общего небольшого корпуса. Поэтому интервалы приводятся как ориентир точности оценки, а не как доказательство превосходства на любых игровых данных.

Таблица 10 — Формализация метрик и проверка воспроизводимости

Конфигурация	Доля совпадений в исходной речи	Интервал Уилсона 95 процентов
Qwen3.5 2B без RAG	2/30, или 6,7 процента	1,8–21,3 процента
Qwen3.5 2B с RAG	20/30, или 66,7 процента	48,8–80,8 процента
GPT-OSS 20B с RAG	29/30, или 96,7 процента	83,3–99,4 процента

Для повторения исследования сначала фиксируются версии Unity, Ollama, модельные теги и квантование. Затем запускаются тесты алгоритмов и доступности endpoints. QA выполняется в заданном порядке без накопления памяти, а каждая попытка сохраняется сразу после получения результата. После окончания запуск скрипта analyze_results.py формирует таблицу по вопросам, сводку и графики. Повторный запуск анализа над теми же JSONL должен дать те же численные агрегаты; повторный inference может дать другие ответы.

Версия кода, на которой выполнены 90 запросов, отмечена в Git тегом experiment-2026-10-30. После эксперимента внесены исправления политики Planning. Они не используются для переинтерпретации прежних ответов как

новых запросов к модели. Исходные протоколы и история изменений позволяют отличить измеренный исходный вариант от окончательной инженерной редакции.

Для проверки корректности корпуса все вопросы относятся к авторским документам. Ни модель, ни человек не могут получить ответ из внешней энциклопедии о вымышленной деревне. Вместе с тем набор не является скрытым тестом: разработчик видел его при подготовке сценариев. Это ограничивает силу вывода и требует нового независимого корпуса перед заявлением о практической точности в другом проекте.

4.8 Ограничения экспериментального вывода

Корпус состоит из 30 коротких вопросов к одному небольшому миру. Он содержит известные и однозначные ответы, но не охватывает длинные переговоры, неоднозначные намерения, многопользовательские события и изменение лора во время разговора. Для обобщения потребуются независимые корпуса и несколько повторов на вопрос.

Модели различаются размером, обучением и способом исполнения. Эксперимент без RAG и с RAG для одной локальной модели лучше изолирует влияние знаний, чем сопоставление локальной и облачной моделей. Последнее одновременно меняет несколько факторов, поэтому его следует рассматривать как практическое сравнение конфигураций.

Автоматическая оценка по вариантам ответа не является полноценной проверкой галлюцинаций. Отказ сообщить неизвестный факт и уверенный неправильный ответ одинаково могут не совпасть с эталоном, хотя имеют разную природу. Частота галлюцинаций в строгом смысле требует отдельной экспертной разметки утверждений, которая в этом эксперименте не подменяется дополнением до 100 процентов.

Тесты действий покрывают встроенные обработчики и явно сформулированные инварианты. Они не доказывают корректность произвольного пользовательского IGameAction. Особенно важен запрет

побочных эффектов Validate: при расширении registry разработчик обязан сохранять этот контракт.

Свободный диалог не гарантирует соблюдение всех мягких характеристик личности. Ограничения на передачу предметов и доступ к сущностям обеспечиваются C#, но художественная последовательность речи требует более широкого набора оценок с участием пользователей. Текущая работа подтверждает инженерный контур, а не завершённую модель социальной симуляции.

Проверка интерфейса включает основные операции: открытие окна, переключение вкладок, создание Terrain, просмотр NPC, знаний и маршрутов, запуск диалога и просмотр trace. Скриншоты подтверждают наблюдавшиеся состояния. Они не заменяют автоматическую проверку и не используются для выведения численных результатов модели.

Итог испытаний должен рассматриваться вместе с исходными XML и JSON. Архитектурная цель — отделить недоверенное решение от состояния Unity — проверяется инвариантами и end-to-end сценарием. Эксперимент над фактами показывает применимость индивидуальной базы знаний в заданном корпусе и одновременно выявляет пределы небольшой модели и бесплатного API.

Ответ на RQ1 ограничен данным корпусом: при одинаковой локальной модели RAG повысил совпадение факта в исходной речи с 6,7 до 66,7 процента, а в принятой реплике — с 3,3 до 53,3 процента. Ответ на RQ2 состоит в измеренных полных медианах 6,33, 6,95 и 30,95 секунды; бесплатная облачная квота существенно входит в пользовательское ожидание. Для RQ3 программные проверки подтвердили сформулированные инварианты на положительных и ошибочных входах, а два реальных прогона продемонстрировали передачу и перемещение. Конечность набора тестов и единичность сквозных прогонов сохраняют ограничения этого вывода.

ЗАКЛЮЧЕНИЕ

В работе разработан WorldWeaver — подключаемый инструментарий Unity для генерации рельефа и контекстно-зависимых неигровых персонажей. Программный результат включает UPM-пакет, окно Editor, типизированные контракты TerrainSpec и NpcDecision, локальный retrieval, явные модельные маршруты и систему проверяемых действий.

Анализ аналогов определил интеграционный характер вклада. Известные методы RAG, процедурной генерации и навигации соединены в переносимом контуре с проверяемыми границами ответственности.

Для генерации мира реализован переход от естественного языка к ограниченной спецификации и далее к алгоритмической heightmap. Языковая модель выбирает параметры, а C# обеспечивает вычисление поверхности и проверку диапазонов. Повторная генерация по seed позволяет исследовать результат отдельно от вероятностного вывода LLM.

Для NPC выделены статические знания, события и разговорная память. Scope ограничивает документы до ранжирования. Embeddings и vector store отделены интерфейсами, что позволяет заменить backend без изменения логики персонажа. Ограниченный контекст исключает необходимость передавать всю историю разговора при каждом запросе.

Предложенные моделью последствия проходят строгий разбор и предварительную проверку. Инвентарь изменяется после проверки всего набора встроенных действий, а перемещение исполняет NavMeshAgent. Это устраняет прямую зависимость состояния сцены от формулировки speech, хотя не гарантирует художественную или смысловую безошибочность любой реплики.

На фиксированном корпусе из 30 вопросов RAG повысил долю совпадений известного факта в исходной речи локальной Qwen3.5 2B с 2/30 до 20/30. Облачная GPT-OSS 20B с RAG дала 29/30, включая один ошибочный API-запрос в общем знаменателе. Для принятой игроком речи

обе RAG-конфигурации показали 16/30: проверки действий отсекли часть фактически подходящих ответов. Эти показатели подтверждают пользу внешних знаний и одновременно демонстрируют, что retrieval и качество исходной реплики недостаточны для оценки целого игрового pipeline.

В Unity 6 успешно пройдены тесты: 56 проверок EditMode и два детерминированных PlayMode-теста. Два отдельных прогона с реальными модельными маршрутами завершили ограниченный переговорный сценарий, передали по одному Crystal и вызвали движение к House. У локального NPC сохранилось смысловое расхождение речи с исполненной передачей. Дополнительный GUI-повтор завершил передачу, но вернул бездействие вместо движения; устойчивость автономного планирования остаётся ограниченной. Для Terrain все 40 парных генераций совпали; медиана алгоритма составила от 2,237 мс при разрешении 65 до 136,130 мс при 513. Чистая установка архива пакета в новый проект завершилась без ошибок компиляции.

Основные ограничения связаны с малым корпусом эксперимента, отсутствием постоянного сохранения памяти, ограниченной выразительностью TerrainSpec и best-effort характером бесплатного cloud API. Голос, импорт PDF и автоматическая навигация по произвольному сгенерированному ландшафту не входят в подтверждённые функции версии. Направления дальнейшей работы включают формализацию дополнительных игровых политик, долговременную память, расширение корпусов оценки и подключение специализированного vector backend при росте объёма знаний.

Созданный инструментарий предоставляет основу для дальнейших исследований и повторного использования в Unity-проектах. Его существенное свойство заключается в сохранении границы между вероятностным выбором модели и детерминированным исполнением правил игрового мира.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Unity Technologies. Unity 6000.3.27f1 Release Notes. — 2026. — URL: https://unity.com/releases/editor/whats-new/6000.3.27f1 (дата обращения: 30.10.2026). — Текст: электронный.

2. Unity Technologies. Git dependencies Unity Manual. — 2022–2026. — URL: https://docs.unity3d.com/2022.3/Documentation/Manual/upm-git.html (дата обращения: 30.10.2026). — Текст: электронный.

3. Unity Technologies. TerrainData SetHeights. — 2026. — URL: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/TerrainData.SetHeights.html (дата обращения: 30.10.2026). — Текст: электронный.

4. Unity Technologies. NavMeshBuilder BuildNavMeshData. — 2026. — URL: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/AI.NavMeshBuilder.BuildNavMeshData.html (дата обращения: 30.10.2026). — Текст: электронный.

5. Unity Technologies. Configure AI Gateway Assistant 2.9. — 2026. — URL: https://docs.unity3d.com/Packages/com.unity.ai.assistant@2.9/manual/integration/ai-gateway-get-started.html (дата обращения: 30.10.2026). — Текст: электронный.

6. Convai. Configure character actions. — 2026. — URL: https://github.com/Conv-AI/Convai-Documentation/blob/main/plugins-and-integrations/convai-unity-sdk/features/character-actions/configuring-actions.md (дата обращения: 30.10.2026). — Текст: электронный.

7. Inworld AI. Character Studio deprecation and Agent Runtime. — 2026. — URL: https://docs.inworld.ai/character-studio-deprecation (дата обращения: 31.10.2026). — Текст: электронный.

8. NVIDIA. NVIDIA ACE for Games. — 2026. — URL: https://developer.nvidia.com/ace-for-games (дата обращения: 30.10.2026). — Текст: электронный.

9. Promethean AI. Promethean AI. — 2026. — URL: https://www.prometheanai.com/ (дата обращения: 30.10.2026). — Текст: электронный.

10. Procedural Worlds. Procedural Worlds. — 2026. — URL: https://www.procedural-worlds.com/ (дата обращения: 30.10.2026). — Текст: электронный.

11. UndreamAI. LLMUnity. — 2026. — URL: https://github.com/undreamai/LLMUnity (дата обращения: 30.10.2026). — Текст: электронный.

12. LangChain. LangChain knowledge base. — 2026. — URL: https://github.com/langchain-ai/docs/blob/main/src/oss/langchain/knowledge-base.mdx (дата обращения: 30.10.2026). — Текст: электронный.

13. LlamaIndex. Using VectorStoreIndex. — 2026. — URL: https://developers.llamaindex.ai/python/framework/module_guides/indexing/vector_store_index/ (дата обращения: 30.10.2026). — Текст: электронный.

14. Lewis P., Perez E., Piktus A. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. — 2020. — URL: https://arxiv.org/abs/2005.11401 (дата обращения: 30.10.2026). — Текст: электронный.

15. Park J. S., O'Brien J. C., Cai C. J. et al. Generative Agents Interactive Simulacra of Human Behavior. — 2023. — URL: https://arxiv.org/abs/2304.03442 (дата обращения: 30.10.2026). — Текст: электронный.

16. Yao S., Zhao J., Yu D. et al. ReAct Synergizing Reasoning and Acting in Language Models. — 2022–2023. — URL: https://arxiv.org/abs/2210.03629 (дата обращения: 30.10.2026). — Текст: электронный.

17. Ollama. Structured outputs Ollama. — 2026. — URL: https://docs.ollama.com/capabilities/structured-outputs (дата обращения: 30.10.2026). — Текст: электронный.

18. Ollama and Qwen. Qwen3.5 2B Ollama model card. — 2026. — URL: https://ollama.com/library/qwen3.5:2b (дата обращения: 30.10.2026). — Текст: электронный.

19. Ollama and Google. EmbeddingGemma Ollama model card. — 2025–2026. — URL: https://ollama.com/library/embeddinggemma (дата обращения: 30.10.2026). — Текст: электронный.

20. Google AI for Developers. EmbeddingGemma inference. — 2026. — URL: https://ai.google.dev/gemma/docs/embeddinggemma/inference-embeddinggemma-with-sentence-transformers (дата обращения: 30.10.2026). — Текст: электронный.

21. OpenAI. Chat API reference. — 2026. — URL: https://developers.openai.com/api/reference/resources/chat (дата обращения: 30.10.2026). — Текст: электронный.

22. Groq. Structured Outputs Groq. — 2026. — URL: https://console.groq.com/docs/structured-outputs (дата обращения: 30.10.2026). — Текст: электронный.

23. Pollinations. Pollinations Classic API. — 2026. — URL: https://github.com/pollinations/pollinations/blob/master/APIDOCS.md (дата обращения: 30.10.2026). — Текст: электронный.

24. Unity Technologies. Unity Test Framework introduction Unity 6.3. — 2026. — URL: https://docs.unity3d.com/6000.3/Documentation/Manual/test-framework/test-framework-introduction.html (дата обращения: 31.10.2026). — Текст: электронный.

25. Unity Technologies. Undo RegisterCompleteObjectUndo. — 2026. — URL: https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Undo.RegisterCompleteObjectUndo.html (дата обращения: 30.10.2026). — Текст: электронный.

26. JSON Schema. JSON Schema Draft 2020 12. — 2020. — URL: https://json-schema.org/draft/2020-12 (дата обращения: 30.10.2026). — Текст: электронный.

27. ВолгГТУ. Бланки учебно методического управления ВолгГТУ. — 2019–2026. — URL: https://www.vstu.ru/obrazovanie/umu/blanki/ (дата обращения: 30.10.2026). — Текст: электронный.

28. Росстандарт. ГОСТ 19.201-78 Техническое задание Требования к содержанию и оформлению. — 1978. — URL: https://protect.gost.ru/gost/details/b64061cf-6bb2-4c75-aee9-18bb5e26e554 (дата обращения: 30.10.2026). — Текст: электронный.

29. Росстандарт. ГОСТ 7.32-2017 Отчет о научно исследовательской работе. — 2017. — URL: https://protect.gost.ru/gost/details/7d280e43-7036-4a69-8e6e-d15867028343 (дата обращения: 30.10.2026). — Текст: электронный.

30. Росстандарт. ГОСТ Р 7.0.100-2018 Библиографическая запись Библиографическое описание. — 2018. — URL: https://protect.gost.ru/gost/details/389dfd5a-535a-458a-81c3-14b729b1cee1 (дата обращения: 30.10.2026). — Текст: электронный.

31. CoplayDev. MCP for Unity. — 2026. — URL: https://github.com/CoplayDev/unity-mcp (дата обращения: 30.10.2026). — Текст: электронный.

32. Shaker N., Togelius J., Nelson M. J. Procedural Content Generation in Games. — 2016. — URL: https://pcgbook.com/ (дата обращения: 30.10.2026). — Текст: электронный.

33. LM Studio. LM Studio OpenAI compatibility. — 2026. — URL: https://lmstudio.ai/docs/developer/openai-compat (дата обращения: 30.10.2026). — Текст: электронный.

34. ggml-org. llama.cpp. — 2026. — URL: https://github.com/ggml-org/llama.cpp (дата обращения: 30.10.2026). — Текст: электронный.

35. Kyzr O. Procedural Generation of Outdoor Scenes Bachelor Thesis. — 2023. — URL: https://dcgi.fel.cvut.cz/en/theses/2023/kyzrondr/ (дата обращения: 31.10.2026). — Текст: электронный.

36. Marcher F. Procedural Terrain Generation Tool in Unity Master Thesis DOI 10.3217/rb5eq-fv649. — 2024. — URL: https://repository.tugraz.at/publications/rb5eq-fv649 (дата обращения: 31.10.2026). — Текст: электронный.

37. Wakeford O. R. J. Procedural Terrain Generation with Fixed Points Bachelor Thesis. — 2025. — URL: https://dspace.cuni.cz/handle/20.500.11956/202622 (дата обращения: 31.10.2026). — Текст: электронный.

38. Aslan P. Understanding LLMs for Game NPCs An Exploration of Opportunities Master Thesis. — 2024. — URL: https://kth.diva-portal.org/smash/get/diva2%3A1938971/FULLTEXT01.pdf (дата обращения: 31.10.2026). — Текст: электронный.

ПРИЛОЖЕНИЕ А Техническое задание

Полная редакция технического задания поставляется отдельным документом «Техническое задание WorldWeaver». Приложение фиксирует связь обязательных требований с реализацией. Приёмка выполняется по согласованным критериям и сохранённым протоколам.

Таблица 11 — ПРИЛОЖЕНИЕ А Техническое задание

Требование	Основные элементы реализации	Доказательство
Terrain по описанию	TerrainSpec, TerrainGenerator, TerrainAuthoring	Тесты разбора, диапазонов и seed
Модельные маршруты	ModelRouter, OrchestratorProfile	Тест назначения разных задач
Локальный inference	LocalProvider, Ollama	Real-provider JSON и PlayMode
Бесплатный cloud	OpenAICompatibleProvider, Pollinations	HTTP-проба и 30 QA-вызовов
Индивидуальный RAG	RagIndex, InMemoryVectorStore	Scope-тест и hit@3
Проверка действий	NpcDecision, ActionDispatcher	Amount, target, permissions и double spend
Перемещение	WorldAction, NavMeshAgent	PlayMode MoveTo/Follow
Диагностика	PipelineTrace, WorldWeaverWindow	Trace JSON и вкладка Debug

ПРИЛОЖЕНИЕ Б Приемочные сценарии

Таблица 12 — ПРИЛОЖЕНИЕ Б Приемочные сценарии

Код	Начальные условия	Действие	Ожидаемое состояние
TC-01	Совместимый Unity-проект	Подключить UPM и открыть Tools	Окно и компоненты доступны
TC-02	TerrainSpec seed 42	Дважды вызвать Generate	Все высоты совпадают
TC-03	resolution 128	Разобрать спецификацию	Контролируемый отказ
TC-04	NPC знает village, guard имеет secret	Retrieval от merchant	Secret не попадает в выдачу
TC-05	NPC Crystal 10, Player 0	GiveItem 1	Остатки 9 и 1
TC-06	Лимит передачи 1	GiveItem 2	Остатки не меняются
TC-07	NPC Crystal 10, лимит 100	Два GiveItem по 6	Пакет отклонён целиком
TC-08	Цели Admin нет в KnownEntities	GiveItem Admin	Отказ Unknown target
TC-09	NPC на NavMesh	MoveTo House	Агент перемещается по пути
TC-10	Агент движется	Follow, затем StopMovement	Following, затем остановка
TC-11	HTTP-сервер возвращает 429, затем 200	Запрос с одним повтором	Ровно два обращения
TC-12	HTTP-сервер не отвечает	Cancellation или timeout	Задача завершается без зависания
TC-13	Реальный Ollama доступен	Диалог и Planning	JSON, проверка и реальные последствия
TC-14	Бесплатный API доступен	Смена Dialogue route	Ответ cloud, Planning остаётся local

Исходный набор QA-сценариев содержит 30 строк в results/scenarios.json. В каждом случае зафиксированы вопрос, ожидаемые варианты и source ID. Файл metrics-per-question.csv связывает эти сценарии с наблюдавшимся ответом и агрегатами. При повторном испытании требуется сохранить версии моделей и условия запуска, а не сравнивать только итоговые проценты.

Дополнительная activity-диаграмма показывает обработку ошибки и успешного ответа. Это схема реализованного pipeline, а не обещание автоматического исправления любого ответа модели.

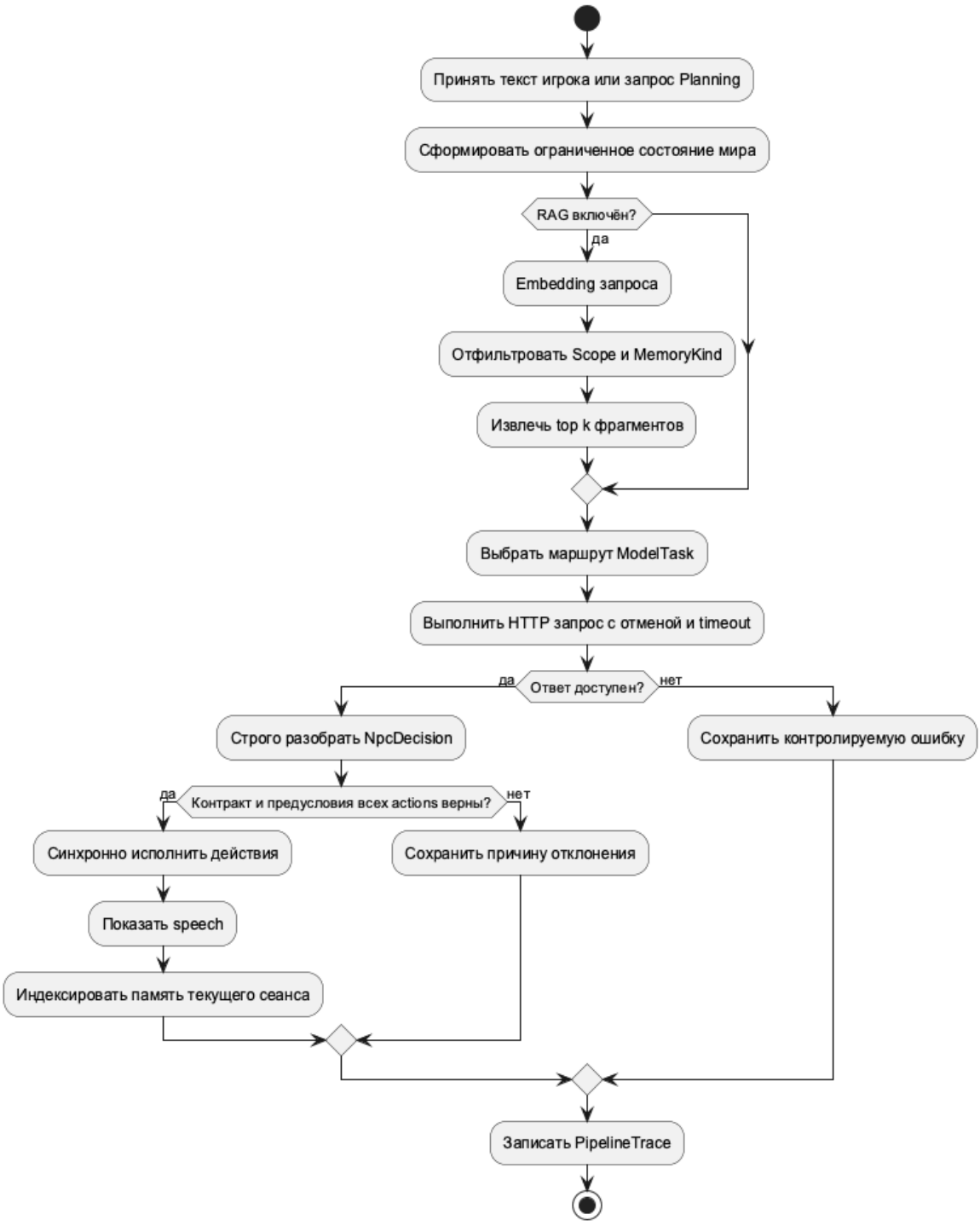


Рисунок 16 — Обработка запроса и отказов pipeline

ПРИЛОЖЕНИЕ В Корпус вопросов к индивидуальным знаниям

Корпус содержит десять тематических документов и по три вопроса к каждому. В таблице приведены автоматические варианты совпадений. Частичные основы имени, например «Эрин», позволяют учитывать падежные формы, но не заменяют экспертную проверку смысла.

Таблица 13 — ПРИЛОЖЕНИЕ В Корпус вопросов к индивидуальным знаниям

ID	Вопрос	Source ID и допустимые варианты
qa-01	Как называется наша деревня?	village: Лунный Брод, Лунном Броде
qa-02	Кто основал деревню?	village: Аэлл
qa-03	На какой реке стоит деревня?	village: Тихая, Тихой
qa-04	Как называется лечебный кристалл?	crystal: Зорецвет
qa-05	Где добывают лечебный кристалл?	crystal: Семи Ветров
qa-06	Какова обычная цена одного кристалла?	crystal: 7, семь
qa-07	Как зовут лекаря деревни?	healer: Эрин
qa-08	Где принимает лекарь?	healer: западн
qa-09	Какое растение используют для настоя от лихорадки?	healer: серебролист
qa-10	Кто был твоим учителем?	merchant: Берен
qa-11	Каких животных ты боишься?	merchant: волк
qa-12	Когда открывается твоя лавка?	merchant: рассвет
qa-13	Где стоит твой дом?	house: восточн
qa-14	Что висит над дверью твоего дома?	house: медный полумесяц, медного полумесяца
qa-15	Что ты обычно делаешь после торговли?	house: осматри, осмотр
qa-16	Как называется деревенский колодец?	well: Зеркальн
qa-17	Какова глубина колодца в локтях?	well: 18, восемнадцать
qa-18	Кто проверяет воду колодца?	well: Тарен
qa-19	В какой день проходит праздник Урожая?	festival: 12, двенадцат
qa-20	Какой символ у праздника Урожая?	festival: белый фонарь, белого фонаря
qa-21	Какую песню поют на празднике?	festival: Семь мостов

ID	Вопрос	Source ID и допустимые варианты
qa-22	Из какого материала построен северный мост?	bridge: красного камня, красный камень
qa-23	Сколько стоит проход по северному мосту?	bridge: 2, две
qa-24	До какого времени разрешён проход по мосту после наводнения?	bridge: полудн
qa-25	Какие отметки обозначают вход в шахту?	mine: три сини, тремя сини
qa-26	Как зовут старшего шахтёра?	mine: Дорин
qa-27	Какая штольня закрыта?	mine: южн
qa-28	Где находится деревенский архив?	library: башне Ив, башня Ив
qa-29	Как зовут хранительницу архива?	library: Сели
qa-30	Как называется самая старая книга в архиве?	library: Карта зимнего неба

ПРИЛОЖЕНИЕ Г Порядок установки и повторения проверки

Для быстрого старта устанавливается Unity 6000.3.27f1 и открывается проект UnityProject. Пакет находится отдельно в Packages/com.worldweaver.toolkit и подключён через локальную file-зависимость. Для установки в другой проект выбирается Window → Package Manager → Add package from tarball и передаётся com.worldweaver.toolkit-1.0.0.tgz. Новый проект должен получить зависимость Newtonsoft JSON 3.2.1. Проверка такого импорта с новой Library сохранена в clean-install.json.

После импорта через Tools → WorldWeaver → Create Demo in New Scene создаётся демонстрация. Она содержит собственные TerrainData, материалы, профили моделей и KnowledgeBase. Повторное создание формирует новые ассеты с уникальными именами, поэтому перед ним следует сохранить текущую сцену. Пример не требует удаления прежних Terrain другого проекта.

Локальный daemon Ollama должен быть доступен по http://127.0.0.1:11434. Для маршрута Terrain необходим qwen3.5:2b, для Dialogue и Planning — qwen2.5-coder:latest, для индексации — embeddinggemma. Имена тегов следует сопоставить с локальным каталогом. Изменение embedding-модели требует перестроения индекса. Внешние модельные веса в поставку исходного кода не включаются.

Для бесплатного диалога выбирается CloudRoutes: Terrain и Planning остаются локальными, Dialogue использует Pollinations Classic и псевдоним openai-fast. В debug проверяется фактическое model из CompletionResult. Одновременно выполнять несколько cloud-сценариев с одного IP не рекомендуется из-за общей квоты. Ключи и платёжные данные для проверенной конфигурации не нужны.

Первый этап повторной проверки включает EditMode и детерминированный PlayMode. Реальные провайдеры проверяются отдельно с флагом WW_REAL_TESTS=1 и фильтром

WorldWeaver.Tests.RealProviderIntegration. Без флага эти тесты пропускаются, поэтому обычный успешный набор без внешней модели нельзя выдавать за подтверждение inference. Перед запуском необходимо завершить другую копию Editor, использующую тот же projectPath.

Для повторения QA применяется ResearchRunner с WW_EVAL_PROVIDER=local либо cloud. Вопросы читаются из scenarios.json; для local выполняются конфигурации без RAG и с RAG, для cloud — с RAG. Накопление памяти выключается. Полученные JSONL следует сохранить в новый каталог или зафиксировать отдельной версией, чтобы исходные результаты не были потеряны.

Скрипт analyze_results.py получает сохранённые JSONL и terrain.json и записывает metrics-per-question.csv, summary.json и графики. Воспроизведение агрегатов требует Python с matplotlib и numpy; оно не требует повторной оплаты API или запуска модели. Для проверки конечного игрового эффекта используются XML и end-to-end JSON, содержащие остатки Inventory и измеренное изменение позиции.

При отказе локального соединения сначала проверяются daemon и наличие модели, затем доступность embedding endpoint. При HTTP 429 либо 402 с Retry-After следует дождаться разрешённого интервала; при HTTP 400 исправляется запрос, поскольку автоматический повтор такого тела обычно бессмысленен. Ошибочный JSON анализируется в Debug и не превращается в действие вручную. При отклонённом MoveTo проверяются положение агента на NavMesh и достижимость целевой точки.